R A D A S

Sistema de gestión para un rooftop bar · Puerto Madero, Buenos Aires

Entrega final · Etapa 3

Aplicación integrada y documentación

Manual Técnico · Manual de Usuario · Base de datos y CRUD · Pruebas unitarias · Pruebas de integración · Instalación y puesta en marcha

Equipo	**Dynamis**
Integrantes	Ximena Facal · Franco Guarachi · Rodrigo Pinto · Mara Skaarup
Institución	IFTS N.º 29
Materia	Práctica Profesionalizante IV: Proyecto Integrador
Docentes	Emir García · Kevin del Bello
Aplicación publicada	**https://radas.onrender.com**
Repositorio	https://github.com/rodgpinto/PPIV-ProyectoIntegrador
Fecha de entrega	Noviembre de 2026

RADAS es un proyecto académico con un bar ficticio. No emite comprobantes con validez fiscal. Todas las capturas de este documento se tomaron del sistema real funcionando con los datos de ejemplo incluidos en el proyecto.

Índice

1 Resumen y acceso rápido 4
- Cómo evaluar el proyecto en 5 minutos 4
- Credenciales de prueba 4

2 Instalación y puesta en marcha 6
- 2.1 Camino 1 · Usar la aplicación publicada 6
- 2.2 Camino 2 · Ejecutar en la computadora (recomendado para evaluar el código) 6
- 2.3 Camino 3 · Ejecutar con un PostgreSQL propio y los scripts SQL 7
- 2.4 Cómo comprobar que la instalación está bien 8
- 2.5 Problemas frecuentes 8

3 Introducción general 10
- 3.1 Qué es RADAS 10
- 3.2 Objetivo 10
- 3.3 Alcance 10
- 3.4 Roles y permisos 10

4 Arquitectura del sistema 12
- 4.1 Modelo general 12
- 4.2 Decisiones de diseño 13
- 4.3 Tecnologías empleadas 14

5 Estructura del proyecto 15
- 5.1 Tamaño y ritmo de cambio 15

6 Base de datos 16
- 6.1 Diagrama Entidad–Relación (DER) 16
- 6.2 Descripción de tablas y campos 17
- 6.3 Relaciones entre tablas 22
- 6.4 Tipos enumerados, restricciones e índices 23

7 Configuración e instalación (referencia técnica) 24
- 7.1 Versiones y dependencias 24
- 7.2 Comandos del proyecto 24
- 7.3 Variables de entorno 24
- 7.4 Crear o restaurar la base de datos 25
- 7.5 Credenciales de prueba 25

8 Componentes principales 26
- 8.1 Núcleo de la API 26
- 8.2 Dominio (lógica pura) 26
- 8.3 Módulos de negocio 27
- 8.4 Flujo completo de un pedido 28

9 Gestión de errores y validaciones 29
- 9.1 Capas de validación 29
- 9.2 Códigos de respuesta 29
- 9.3 Errores de conexión y situaciones excepcionales 29

10 Pruebas implementadas (resumen) 31

11 Manual de Usuario · Introducción y requisitos 32
- 11.1 Para qué sirve RADAS 32
- 11.2 Requisitos mínimos 32
- 11.3 Cómo entrar 32

12 Pantalla principal y menús 34
12.1 Qué hace cada opción 34
13 Flujos de uso paso a paso 36
13.1 Tomar un pedido (camarero, barra, cajero) 36
13.2 Preparar los pedidos: cocina y barra 37
13.3 Entregar, imprimir la cuenta y cobrar 38
13.4 Pasar una cuenta o productos a otra mesa 39
13.5 Ver y marcar mesas (host) 40
13.6 Registrar el ingreso y el egreso del turno (todo el personal) 40
13.7 Registrar una merma 41
13.8 Cobrar y cerrar la caja de la noche (cajero y administrador) 41
13.9 Administrar la carta, los precios y las recetas (administrador) 42
13.10 Cargar mercadería y controlar el stock (administrador) 43
13.11 Empleados, horas y claves (administrador) 45
13.12 Ver ventas e informes (administrador) 45
13.13 Copias de seguridad, auditoría y carta con QR (administrador) 45
13.14 Cambiar mi contraseña 46
14 Mensajes y alertas frecuentes 47
15 Base de datos y CRUD 48
15.1 Cobertura de las operaciones CRUD 48
15.2 Validaciones implementadas 48
15.3 Evidencia 1 · CRUD desde la pantalla (Proveedores) 49
15.4 Evidencia 2 · Las mismas operaciones contra la API, con las respuestas reales 51
15.5 Evidencia 3 · Prueba automática del CRUD 51
16 Pruebas unitarias 52
16.1 Qué son y cómo se ejecutan 52
16.2 Prueba U1 · Asignación de stock por vencimiento (FEFO) 52
16.3 Prueba U2 · Costos, márgenes y precios 53
16.4 Prueba U3 · Arqueo de caja 54
16.5 Prueba U4 · Validación de números escritos a la argentina 54
16.6 Evidencia de ejecución de las 43 pruebas unitarias 55
16.7 Resultado de toda la batería en los dos motores 56
17 Pruebas de integración 57
17.1 Qué son y cómo se ejecutan 57
17.2 Prueba I1 · Un pedido impacta en el stock, la mesa y el cobro 57
17.3 Prueba I2 · CRUD completo: crear, leer, actualizar y eliminar con integridad 59
17.4 Prueba I3 · Cierre de caja por canal 61
17.5 Prueba I4 · Recorrido completo en un navegador real (extremo a extremo) 61
17.6 Evidencia de ejecución de las pruebas de integración (62 casos de los archivos documentados) 62
18 Entregables, verificación y limitaciones conocidas 64
18.1 Lista de entregables 64
18.2 Qué se verificó y qué no 64
18.3 Guías complementarias del repositorio 64

Cómo está organizado este documento: capítulos 1–2 (acceso rápido e instalación) · **Parte I, Manual Técnico**: capítulos 3–10 · **Parte II, Manual de Usuario**: capítulos 11–14 · **Parte III, Base de datos, CRUD y pruebas**: capítulos 15–17 · capítulo 18 (entregables).

1 Resumen y acceso rápido

RADAS es una aplicación web que resuelve la operación diaria de un bar: toma de pedidos por mesa y barra, pantallas de cocina y barra, cobro y cierre de caja, control de stock con vencimientos, costos y márgenes por receta, horas del personal, informes para el dueño, auditoría y copias de seguridad. Se usa desde el navegador de una tablet, un celular o una computadora; no hay que instalar nada en esos dispositivos.

Cómo evaluar el proyecto en 5 minutos

Si querés...	Hacé esto	Capítulo
Probar la aplicación sin instalar nada	Abrí **https://radas.onrender.com** (la primera visita puede tardar cerca de un minuto: el servicio gratuito se «duerme» sin uso) e ingresá con las credenciales de evaluación de la tabla de abajo.	2.1
Ejecutarla en tu computadora	Con Node.js 22.5 o más nuevo: `npm install`, `npm run seed`, `npm run dev` y abrir `http://localhost:3000`. No hace falta instalar ninguna base de datos.	2.2
Usar tu propio PostgreSQL	Crear una base vacía y ejecutar `sql/schema.sql` y `sql/datos-ejemplo.sql`; después apuntar la aplicación con `DATABASE_URL`.	2.3
Comprobar que todo funciona	`npm test`: 187 pruebas automáticas en unos 10 segundos, sin base de datos instalada.	2.4
Entender cómo está hecho	Manual Técnico: arquitectura, estructura, base de datos con DER, componentes, errores y pruebas.	3 a 10
Aprender a usarla	Manual de Usuario: pantallas, flujos paso a paso con capturas y mensajes frecuentes.	11 a 14
Ver el CRUD y las pruebas con evidencia	Capítulos de Base de datos y CRUD, Pruebas unitarias y Pruebas de integración.	15 a 17

Credenciales de prueba

Todos los usuarios de los datos de ejemplo (instalación local y script SQL) tienen la misma clave: `radas1234`. Cada usuario tiene un rol distinto, así que conviene ingresar con varios para ver cómo cambia el menú y qué puede hacer cada uno.

Email	Rol	Qué ve / qué hace
`admin@radas.test`	Administrador (dueño)	Todo: carta y precios, insumos, stock, informes, caja, empleados, auditoría y copias.
`cajero@radas.test`	Cajero	Pedidos, cobro, caja de la noche y cierre.
`camarero@radas.test`	Camarero	Toma pedidos, entrega, cobra, pasa cuentas entre mesas.
`barra@radas.test`	Barra	Pantalla de barra, pedidos del canal barra.
`cocina@radas.test`	Cocina	Pantalla de cocina.

Email	Rol	Qué ve / qué hace
`host@radas.test`	Host	Disponibilidad de mesas (ocupada / libre).

> **Aplicación publicada (https://radas.onrender.com).** Para evaluar, ingresar con **admin@radas.test** y la clave **radas1234** (rol administrador: ve todos los módulos). Los otros usuarios de la tabla se pueden usar para ver cómo cambia el menú según el rol. Son datos de demostración: no hay información real.

2 Instalación y puesta en marcha

Esta sección está escrita para que cualquier docente pueda descargar el proyecto, configurarlo y ejecutarlo **siguiendo únicamente este documento**. Hay tres caminos; el más simple es el 2.2.

Camino	Necesita	Base de datos	Dificultad
2.1 Aplicación publicada	Solo un navegador	Ya está (PostgreSQL en la nube)	Ninguna
2.2 Local con Node.js	Node.js 22.5+ y conexión a internet para `npm install`	Incluida (PGlite, PostgreSQL embebido)	Baja
2.3 Local con PostgreSQL propio	Node.js 22.5+ y PostgreSQL (se probó con la 16)	Se crea con los scripts `sql/*.sql`	Media

2.1 Camino 1 · Usar la aplicación publicada

1. Abrir **https://radas.onrender.com** en cualquier navegador (Chrome, Edge, Firefox o Safari; en computadora, tablet o celular).
2. **Esperar** si hace falta: el plan gratuito de Render suspende el servicio tras 15 minutos sin visitas y un monitor de UptimeRobot lo consulta cada 5 minutos para evitarlo, pero si estuviera dormido tarda cerca de un minuto en despertar. Si la página no responde de inmediato, alcanza con esperar y recargar.
3. Ingresar con el usuario de evaluación indicado en el capítulo 1.
4. Comprobaciones rápidas: `https://radas.onrender.com/api/salud` debe responder `{"ok":true}`, y `https://radas.onrender.com/carta` muestra la carta pública para clientes (sin login), la que se abre al escanear el QR.

> La aplicación publicada corre en Render (servidor) con la base en Neon (PostgreSQL), y una acción de GitHub hace una copia de la base cada noche. Es un entorno de demostración gratuito: no está pensado para operar un bar real todas las noches.

2.2 Camino 2 · Ejecutar en la computadora (recomendado para evaluar el código)

Requisitos

Requisito	Versión	Cómo comprobarlo
Node.js	**22.5 o más nuevo** (se desarrolló y probó con 22.22.0)	`node -v`
npm	el que trae Node.js	`npm -v`
Sistema operativo	Windows 10/11, macOS o Linux	—
Espacio en disco	unos 300 MB con las dependencias	—

Requisito	Versión	Cómo comprobarlo
Conexión a internet	solo para `npm install` (descarga de dependencias)	—

Si no tiene Node.js, descargarlo de **nodejs.org** (versión LTS, que hoy es 22 o superior) y reiniciar la terminal.

Pasos

1. **Obtener el proyecto.** Descomprimir el archivo `.zip` entregado, o clonar el repositorio: `git clone <URL-del-repositorio>`. Entrar a la carpeta: `cd radas` (el nombre de la carpeta puede variar).
2. **Instalar dependencias** (una sola vez): `npm install`. Instala React, esbuild (para compilar la interfaz) y PGlite, entre otras.
3. **Cargar los datos de ejemplo**: `npm run seed`. Crea la base local en la carpeta `.data/pglite` con 6 usuarios, carta, insumos con lotes, mesas, fichajes, cuentas abiertas y ventas simuladas de las últimas noches.
4. **Iniciar la aplicación**: `npm run dev`. Compila la interfaz y arranca el servidor.
5. **Abrir** `http://localhost:3000` en el navegador e ingresar con alguno de los usuarios del capítulo 1 (clave `radas1234`).

```
# Resumen (idéntico en Windows PowerShell, macOS y Linux)
npm install
npm run seed
npm run dev
# → "RADAS escuchando en http://localhost:3000"
```

Operaciones útiles

Quiero...	Comando
Empezar de cero (borrar todos los datos locales)	Detener el servidor, borrar la carpeta `.data` y volver a correr `npm run seed`.
Cambiar el puerto	Linux/macOS: `PORT=3100 npm run dev` · PowerShell: `$env:PORT=3100; npm run dev`
Probar la versión «de producción» de la interfaz	`npm run build` y luego `npm start`
Ver la carta pública local	`http://localhost:3000/carta`
Ver usuarios de ejemplo	Se imprimen al terminar `npm run seed` (todos con clave `radas1234`)

> **Con la base local no hace falta instalar PostgreSQL:** sin la variable `DATABASE_URL` la aplicación usa PGlite, que es PostgreSQL real compilado a WebAssembly y ejecutado dentro de Node.js. El esquema es el mismo que en producción.

2.3 Camino 3 · Ejecutar con un PostgreSQL propio y los scripts SQL

Para quien prefiera revisar la base con sus propias herramientas (psql, pgAdmin, DBeaver). Se verificó con PostgreSQL 16; versiones cercanas deberían funcionar.

1. **Crear la base vacía**: `createdb radas` (o desde pgAdmin: «Create → Database» con el nombre `radas`).
2. **Crear las tablas**: `psql -d radas -f sql/schema.sql`. El script es idempotente: se puede ejecutar más de una vez sin romper nada.
3. **Cargar los datos de ejemplo** (opcional pero recomendado): `psql -d radas -f sql/datos-ejemplo.sql`. Incluye los 6 usuarios de ejemplo (clave `radas1234`).
4. **Instalar y compilar la aplicación**: `npm install` y `npm run build`.
5. **Indicar la base y arrancar**:

```
# Linux / macOS
export DATABASE_URL="postgres://usuario:clave@localhost:5432/radas"
npm start

# Windows PowerShell
$env:DATABASE_URL="postgres://usuario:clave@localhost:5432/radas"
npm start
```

Abrir `http://localhost:3000`. **No** definir `NODE_ENV=production` en este camino: en modo producción la aplicación exige `ADMIN_EMAIL` y `ADMIN_PASSWORD` y no permite cargar datos de ejemplo (por seguridad).

Restaurar una copia de seguridad (alternativa a los scripts SQL)

El sistema genera copias completas en un archivo `.json.gz` (desde **Sistema → Copias de seguridad**). Para restaurar una en otra instalación:

```
node scripts/respaldo.js verificar  ruta/a/copia.json.gz        # solo comprueba el archivo
node scripts/respaldo.js restaurar  ruta/a/copia.json.gz        # muestra qué haría
node scripts/respaldo.js restaurar  ruta/a/copia.json.gz --si   # lo hace (reemplaza TODO)
```

2.4 Cómo comprobar que la instalación está bien

1. La terminal muestra `RADAS escuchando en http://localhost:3000` y el navegador muestra la pantalla de ingreso.
2. Ingresar como `admin@radas.test`: aparece el **Resumen** con tarjetas de stock, caja, cuentas abiertas y salón.
3. Correr las pruebas automáticas en otra terminal: `npm test`. Debe terminar con `pass 187` y `fail 0` (ver el capítulo 16).

2.5 Problemas frecuentes

Síntoma	Causa probable	Solución
`Falta PGlite: ejecutá "npm install"…`	No se instalaron las dependencias	Ejecutar `npm install` en la carpeta del proyecto.
`npm` o `node` «no se reconoce como comando» (Windows)	Node.js no está instalado o la terminal se abrió antes de instalarlo	Instalar Node.js LTS y abrir una terminal nueva.
Errores raros al arrancar o al correr `npm test` (funciones o módulos que no existen)	Node.js anterior a 22.5	Verificar con `node -v` y actualizar a Node.js 22.5 o superior.

Síntoma	Causa probable	Solución
`EADDRINUSE` (puerto en uso)	Otro programa usa el puerto 3000	Usar otro puerto: `PORT=3100` (ver 2.2).
`Falta DATABASE_URL: en producción hace falta PostgreSQL.`	`NODE_ENV=production` sin base configurada	Definir `DATABASE_URL` o quitar `NODE_ENV=production` en desarrollo.
`No se pudo conectar a la base de datos`	Datos de `DATABASE_URL` incorrectos o PostgreSQL apagado	Verificar usuario, clave, host y que el servicio esté activo.
«Email o contraseña incorrectos»	Se corrió `seed` en otra carpeta, o se escribió mal	Usar `radas1234` con los emails del capítulo 1; o borrar `.data` y repetir `npm run seed`.
La página se ve vieja tras actualizar	Caché del navegador	Recargar sin caché (Ctrl + F5).
La aplicación publicada tarda en responder	El plan gratuito de Render se suspende tras 15 min sin visitas	Esperar cerca de un minuto y recargar.
«Demasiados intentos fallidos»	Más de 5 ingresos incorrectos seguidos (protección)	Esperar unos minutos o reiniciar el servidor local.

3 Introducción general

3.1 Qué es RADAS

RADAS es el sistema de gestión de un rooftop bar ficticio ubicado en Puerto Madero (aforo de 32 personas, atención de martes a domingo de 19:00 a 02:00), desarrollado por el equipo Dynamis como proyecto integrador del IFTS N.º 29. Reúne en una sola aplicación web lo que un bar necesita de lunes a domingo: vender, preparar, cobrar, comprar, controlar el stock, saber cuánto se gana y dejar constancia de lo que hace cada persona.

3.2 Objetivo

- Que el personal trabaje de pie, desde una **tablet o un celular**, sin instalar nada: abre una dirección web y entra con su usuario.
- Que el dueño conozca **costos, márgenes y rentabilidad reales** de cada producto, y sepa qué comprar y qué se está desperdiciando.
- Que la información **no dependa de la computadora de nadie**: servidor siempre encendido y copias de seguridad automáticas.
- Que otro grupo pueda **heredar y continuar** el proyecto: código modular, guías y pruebas automáticas.

3.3 Alcance

Incluye	No incluye (todavía)
• Usuarios y 6 roles con permisos controlados en el servidor • Carta, precios, costos y márgenes por receta • Insumos, proveedores, stock por lotes con vencimientos (FEFO), mermas, conteos • Pedidos por mesa y barra, pantalla de cocina/barra (KDS), cobro, ticket no fiscal • Mover cuentas y productos entre mesas • Caja por canal con arqueo de efectivo • Horas del personal (fichajes) • Informes del dueño: ventas, rentabilidad, comparación, consumo real vs teórico, compras sugeridas • Auditoría, copias de seguridad y restauración • Carta pública para clientes con código QR	• Pedidos hechos por el propio cliente desde el QR (hoy solo muestra la carta) • Integración con Mercado Pago o terminales de pago (se anota el medio de pago; el cobro se hace por fuera) • Dividir una cuenta entre varios medios de pago y propinas • Facturación electrónica (ARCA): hay un diseño documentado en `docs/FACTURACION.md`, no implementado • Reportes en PDF (hay descargas en CSV) • Tiempo real con WebSockets (las pantallas se actualizan cada 5 a 10 segundos)

3.4 Roles y permisos

Los permisos se aplican **en el servidor** (cada ruta de la API declara qué roles pueden usarla); ocultar un botón en la pantalla es solo comodidad. Los costos y márgenes nunca viajan a quien no es administrador.

Función	Admin	Cajero	Barra	Camarero	Host	Cocina
Ver la carta, fichar, ver mesas	●	●	●	●	●	●

Función	Admin	Cajero	Barra	Camarero	Host	Cocina
Marcar mesas libres/ocupadas	●	●	—	●	●	—
Tomar pedidos, entregar y cobrar	●	●	●	●	—	—
Pantalla de cocina / barra	ambas	—	barra	—	—	cocina
Pasar cuentas o productos entre mesas	●	●	●	●	—	—
Caja de la noche y cierre	●	●	—	—	—	—
Registrar mermas	●	—	●	●	—	●
Anular un pedido ya preparado, reabrir caja	●	—	—	—	—	—
Productos, precios, insumos, proveedores, stock	●	—	—	—	—	—
Ventas, informes, horas del personal, empleados	●	—	—	—	—	—
Auditoría, copias de seguridad, QR de la carta	●	—	—	—	—	—

4 Arquitectura del sistema

4.1 Modelo general

RADAS sigue una arquitectura **modular por capas**, desplegada como **un solo servicio Node.js** que entrega la API, la interfaz web y la carta pública. Las capas, de arriba hacia abajo:

Capa	Dónde	Responsabilidad
Presentación	`web/`	Interfaz de una sola página (SPA) en React: una pantalla por archivo, componentes comunes y cliente de la API. Se compila a un JS y un CSS estáticos.
Transporte / API	`src/http/`, `src/api.js`	Servidor HTTP nativo de Node, entrega de archivos estáticos con gzip y ETag, y un **despachador** que recorre una tabla de rutas (74 rutas) aplicando autenticación, permiso por rol, validación y auditoría.
Módulos de negocio	`src/modules/<área>/`	Por cada área, un archivo de rutas (permiso + validación + llamada) y uno de servicio (consultas y transacciones). Áreas: auth, catálogo, stock, pedidos, ventas, mesas, fichajes, usuarios, informes, respaldo, auditoría.
Dominio	`src/domain/`	Lógica **pura**, sin base de datos ni HTTP: costos y márgenes, FEFO, alertas, horas, estados de pedido, ventas, informes, fechas, QR. Es lo más fácil de probar y la interfaz importa las mismas funciones.
Persistencia	`src/db.js`, `sql/schema.sql`	Una interfaz única (`db.query`, `db.transaction`) sobre el driver `pg` (producción) o PGlite (desarrollo). El esquema es idempotente y se aplica en cada arranque.

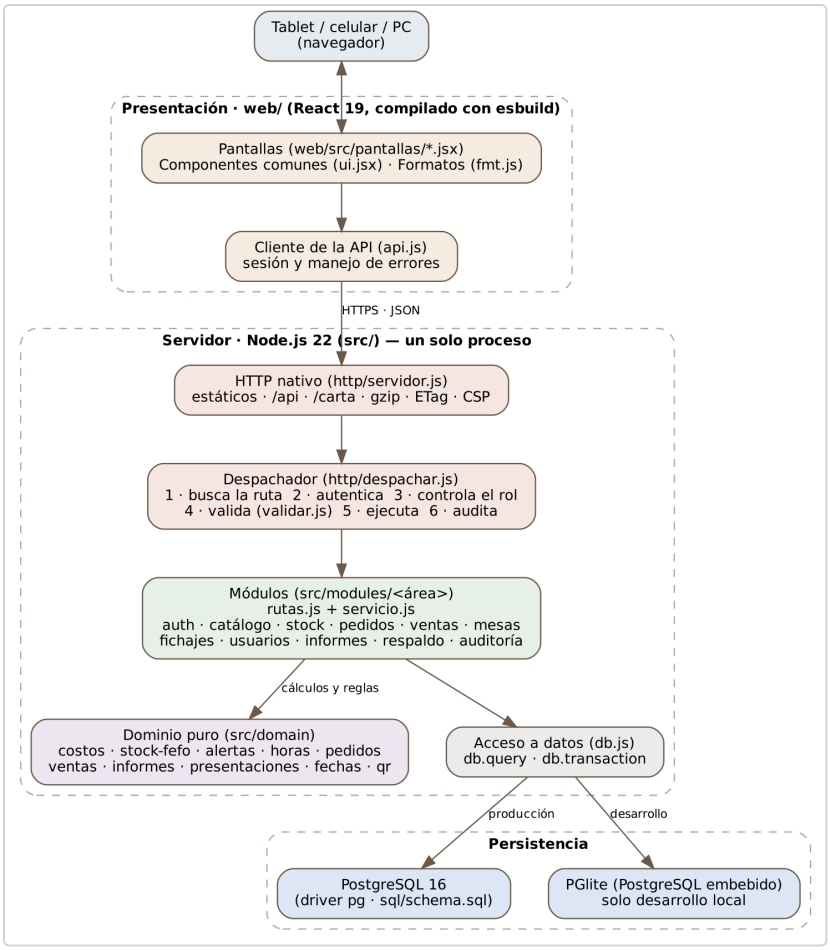


Figura 4.1. Capas y componentes. Una petición entra por HTTP, el despachador la autentica y controla el permiso, el módulo ejecuta la regla de negocio (apoyándose en el dominio puro) y guarda en PostgreSQL dentro de una transacción.

4.2 Decisiones de diseño

- **Sin framework de servidor (sin Express).** Una sola dependencia de ejecución (`pg`). Menos que aprender, menos que actualizar y menos que se rompa; además el despachador recibe un «pedido» simple y toda la API se prueba sin levantar el servidor HTTP.
- **Lógica de negocio en funciones puras.** Cuánto cuesta una receta, qué vence primero, cómo se calcula el arqueo: se prueban sin base de datos (pruebas unitarias del capítulo 16).
- **Transacciones para todo lo que toca más de una tabla.** Un pedido que descuenta stock, un cobro, una restauración de copia: o se hace todo o no cambia nada.
- **Datos históricos «congelados».** Cada línea de pedido guarda el precio y el costo del momento: cambiar la carta después no altera lo ya vendido.
- **Baja lógica por defecto, borrado real solo sin historial.** Lo que tiene ventas, stock o auditoría se da de baja (`activo = FALSE`) para no romper informes ni la integridad referencial (ver capítulo 15).
- **Consulta periódica en lugar de WebSockets.** Las pantallas de cocina y mesas consultan cada 5 a 10 segundos: gasta poco y alcanza para unas 60 personas por noche.
- **Esquema idempotente.** `sql/schema.sql` usa `CREATE … IF NOT EXISTS`: se puede aplicar en cada arranque, así actualizar es reiniciar.
- **Seguridad.** Contraseñas con scrypt, sesiones con token aleatorio guardado como hash y revocables, cabeceras de seguridad y CSP estricta, límite de cuerpo de 100 KB, límite de intentos fallidos de ingreso y validación de toda entrada.

4.3 Tecnologías empleadas

Componente	Tecnología	Versión	Para qué
Entorno de ejecución	Node.js	≥ 22.5 (probado con 22.22.0)	Servidor, scripts y pruebas (`node:test`)
Base de datos	PostgreSQL	16 (probado)	Persistencia en producción
Driver de base	`pg`	^8.12.0	Conexión a PostgreSQL (única dependencia de ejecución)
Base embebida (desarrollo)	PGlite	^0.3.0	PostgreSQL dentro de Node para correr sin instalar nada
Interfaz	React / React DOM	^19.2.0	Pantallas del sistema
Compilación	esbuild	^0.27.0	Empaquetar la interfaz (≈108 KB de JS comprimido)
Pruebas	`node:test` + SQLite en memoria / PostgreSQL	incluido en Node	187 pruebas automáticas
Pruebas de navegador	Playwright + Chromium	opcional	62 pasos de punta a punta
Despliegue gratuito	Render + Neon + GitHub Actions	—	Servidor, base PostgreSQL y copias nocturnas

5 Estructura del proyecto

```
radas/
├── package.json            Dependencias y comandos (dev, seed, build, start, test, respaldo)
├── render.yaml             Despliegue en Render (infraestructura como código)
├── Dockerfile · docker-compose*.yml · deploy/   Alternativas de despliegue (servidor propio)
├── sql/
│   ├── schema.sql          Esquema completo e idempotente (tablas, tipos, índices)
│   └── datos-ejemplo.sql   Datos de demostración para PostgreSQL común
├── src/                    SERVIDOR
│   ├── server.js           Arranque: conecta la base, crea tablas, asegura el admin, copias, abre el
puerto
│   ├── config.js           Todas las variables de entorno en un solo lugar
│   ├── db.js               Acceso a la base (pg o PGlite): db.query / db.transaction
│   ├── api.js              Tabla con las 74 rutas de la API
│   ├── roles.js            Qué rol puede qué (grupos de permisos)
│   ├── bootstrap.js        Preparación de la base y primer administrador
│   ├── seed-demo.js        Datos de ejemplo
│   ├── http/               servidor.js · despachar.js · validar.js · errores.js · carta-publica.js
│   ├── domain/             Lógica pura: costos, stock-fefo, alertas, horas, pedidos, ventas,
│   │                       informes, presentaciones, fechas, auditoria, qr
│   └── modules/            auth · catalogo · stock · pedidos · ventas · mesas · fichajes ·
│                           usuarios · informes · respaldo · auditoria   (rutas.js + servicio.js)
├── web/                    INTERFAZ
│   ├── src/app.jsx         Menú y rutas de pantallas (una entrada por pantalla y rol)
│   ├── src/pantallas/      pedidos · kds · caja · ventas · informes · productos · insumos · stock ·
│   │                       proveedores · mesas · fichar · personal · usuarios · sistema · ticket …
│   ├── src/ui.jsx · fmt.js · api.js    Componentes comunes, formatos (pesos, fechas) y cliente de la
API
│   └── estilos.css         Todos los estilos (incluye impresión del ticket)
├── scripts/                seed.js (datos de ejemplo) · build-web.js · respaldo.js (copias por
terminal)
├── test/                   Pruebas automáticas (*.test.js) y test/e2e/navegador.mjs (Playwright)
└── docs/                   Guías: DESPLIEGUE · COPIAS · MANTENIMIENTO · FACTURACION · este PDF
```

5.1 Tamaño y ritmo de cambio

Parte	Líneas aprox.	Observación
Servidor (`src/`)	≈ 4.500	JavaScript moderno (módulos ES), sin transpilar
Interfaz (`web/src/`)	≈ 3.400	React con hooks, sin librerías de UI externas
Esquema SQL	≈ 260	15 tablas, 11 tipos enumerados, 14 índices
Pruebas (`test/`)	187 casos	Dominio, API con base real y navegador

6 Base de datos

6.1 Diagrama Entidad–Relación (DER)

15 tablas agrupadas por área. «Pata de gallo» = lado «muchos»; círculo = relación opcional; líneas grises = referencias a `usuario`. Se muestran claves y campos principales; el detalle está en 6.2.

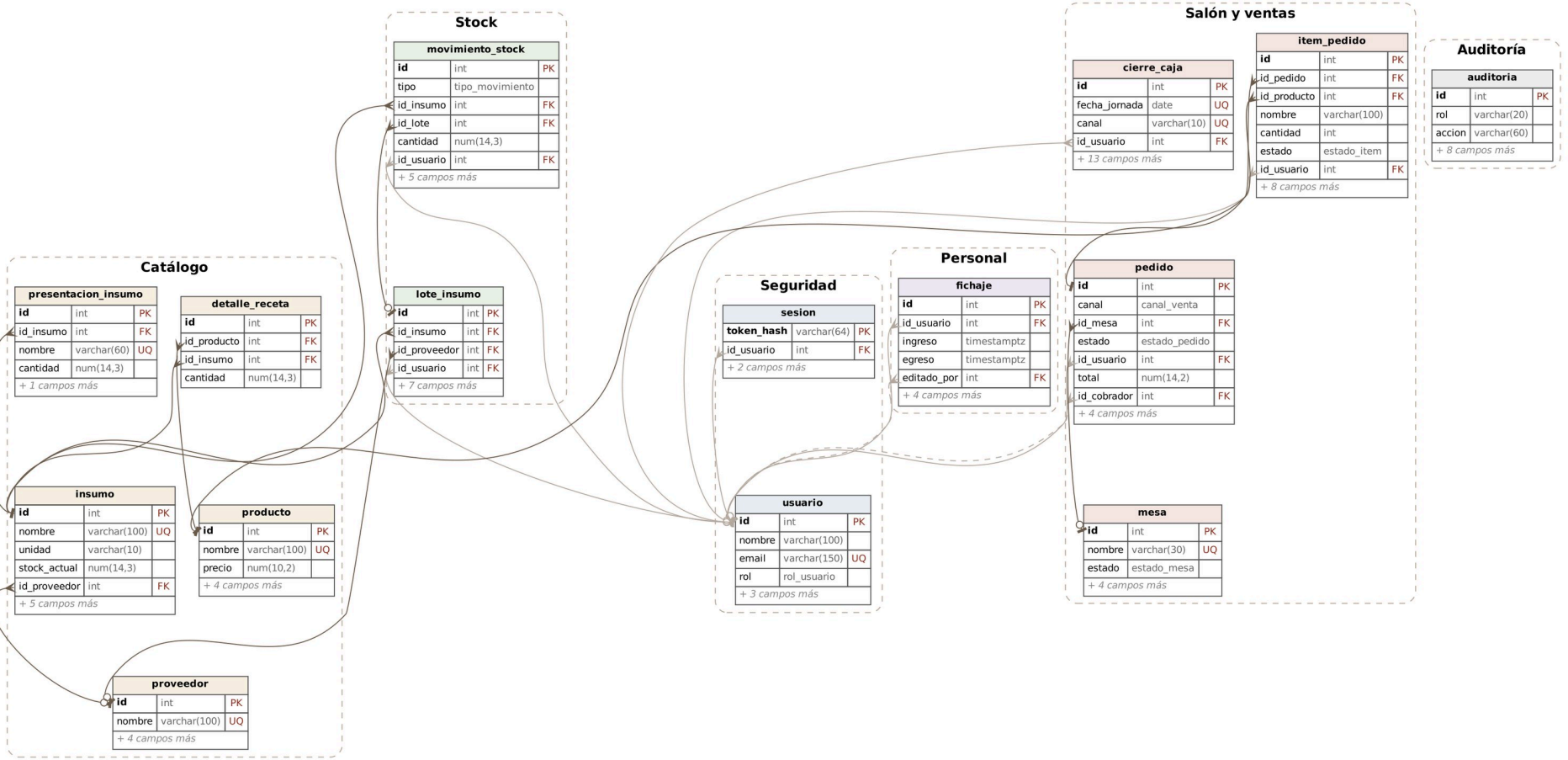


Figura 6.1. DER de RADAS generado a partir del esquema real de PostgreSQL (`sql/schema.sql`). PK = clave primaria · FK = clave foránea · UQ = valor único.

6.2 Descripción de tablas y campos

Tipos tal como los crea PostgreSQL. «NN» = obligatorio (NOT NULL). Todas las tablas tienen `id` autoincremental como clave primaria, salvo `sesion` (clave: `token_hash`).

usuario

Personas que ingresan al sistema (empleados y dueño) con su rol.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
nombre	`VARCHAR(100)`	No			Nombre para mostrar
email	`VARCHAR(150)`	No		único	Email de ingreso (único)
contrasena	`VARCHAR(200)`	No			Contraseña cifrada con scrypt (nunca en claro)
rol	`rol_usuario`	No			administrador, cajero, barra, camarero, host o cocina
activo	`BOOLEAN`	No	true		FALSE = dado de baja (no puede ingresar)
creado_en	`TIMESTAMPTZ`	No	now()		Fecha de alta

sesion

Sesiones abiertas. Se guarda solo el hash del token, nunca el token.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
token_hash	`VARCHAR(64)`	No		PK	Hash del token de sesión (clave primaria)
id_usuario	`INT`	No		FK → usuario (borrado en cascada)	Dueño de la sesión; se borra con el usuario
creada_en	`TIMESTAMPTZ`	No	now()		Inicio de la sesión
expira	`TIMESTAMPTZ`	No			Vencimiento (14 h por defecto)

proveedor

A quién se le compra mercadería.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
nombre	`VARCHAR(100)`	No		único	Nombre (único)
contacto	`VARCHAR(100)`	No	vacío		Persona de contacto
telefono	`VARCHAR(40)`	No	vacío		Teléfono
notas	`VARCHAR(300)`	No	vacío		Observaciones
activo	`BOOLEAN`	No	true		FALSE = dado de baja

producto

Ítems de la carta que se venden (tragos, platos) con su precio.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
nombre	`VARCHAR(100)`	No		único	Nombre en la carta (único)
descripcion	`TEXT`	No	vacío		Texto para la carta de clientes
categoria	`VARCHAR(50)`	No			Clásicos, Tapeo, etc.
precio	`NUMERIC(10,2)`	No			Precio de venta en pesos (≥ 0)
destino	`destino_producto`	No			barra o cocina: decide quién lo prepara
activo	`BOOLEAN`	No	true		FALSE = fuera de la carta (se conserva el historial)

insumo

Ingredientes y bebidas que consumen las recetas; guarda stock y alertas.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
nombre	`VARCHAR(100)`	No		único	Nombre (único)
unidad	`VARCHAR(10)`	No			Unidad base: ml, g o unidad
costo_unitario	`NUMERIC(14,4)`	No	0		Costo por unidad base (último precio de compra)
stock_actual	`NUMERIC(14,3)`	No	0		Cantidad en depósito (suma de los lotes)
stock_minimo	`NUMERIC(14,3)`	No	0		Por debajo se avisa «reponer»
stock_maximo	`NUMERIC(14,3)`	Sí			Por encima se avisa «sobrestock» (opcional)
dias_aviso_vencimiento	`INT`	No	7		Días de anticipación para el aviso «por vencer»
id_proveedor	`INT`	Sí		FK → proveedor	Proveedor habitual (opcional)
activo	`BOOLEAN`	No	true		FALSE = dado de baja

detalle_receta

Receta: cuánto de cada insumo lleva un producto.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
id_producto	`INT`	No		FK → producto, único (con otro campo)	Producto al que pertenece
id_insumo	`INT`	No		FK → insumo, único (con otro campo)	Insumo que usa
cantidad	`NUMERIC(14,3)`	No			Cantidad de insumo por porción, en su unidad base (> 0)

presentacion_insumo

Envases de un insumo («Botella 750 ml», «Caja x 24») para cargar y contar.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
id_insumo	`INT`	No		FK → insumo, único (con otro campo)	Insumo
nombre	`VARCHAR(60)`	No		único (con otro campo)	Nombre del envase (único por insumo)
cantidad	`NUMERIC(14,3)`	No			Cuánto trae un envase, en unidad base (> 0)
principal	`BOOLEAN`	No	false		La que se usa para mostrar «3 botellas + 200 ml»

lote_insumo

Cada ingreso de mercadería: cantidad, costo, vencimiento. Base del FEFO.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
id_insumo	`INT`	No		FK → insumo	Insumo
id_proveedor	`INT`	Sí		FK → proveedor	A quién se compró (opcional)
origen	`VARCHAR(10)`	No	'compra'		compra o ajuste
cantidad_inicial	`NUMERIC(14,3)`	No			Cantidad que entró (> 0)
cantidad_actual	`NUMERIC(14,3)`	No			Cantidad que queda (≥ 0)
costo_unitario	`NUMERIC(14,4)`	No	0		Costo por unidad base de este lote
fecha_ingreso	`DATE`	No			Fecha de entrada
fecha_vencimiento	`DATE`	Sí			Vencimiento (vacío = no vence)
id_usuario	`INT`	Sí		FK → usuario	Quién lo cargó
creado_en	`TIMESTAMPTZ`	No	now()		Momento del registro

movimiento_stock

Historial de todo lo que entra o sale del depósito (ingresos, ventas, mermas, ajustes, reversiones).

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
fecha_hora	`TIMESTAMPTZ`	No	now()		Momento del movimiento
tipo	`tipo_movimiento`	No			ingreso, venta, merma, ajuste o reversion
id_insumo	`INT`	No		FK → insumo	Insumo afectado
id_lote	`INT`	Sí		FK → lote_insumo	Lote del que salió o al que entró
cantidad	`NUMERIC(14,3)`	No			Cantidad (positiva entra, negativa sale)
costo_unitario	`NUMERIC(14,4)`	Sí			Costo al momento
motivo	`motivo_merma`	Sí			Solo mermas: vencido, devolucion, se_cayo, consumo_interno

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
referencia	`VARCHAR(60)`	Sí			Origen, p. ej. item:123 (línea de pedido)
nota	`VARCHAR(300)`	Sí			Comentario libre
id_usuario	`INT`	Sí		FK → usuario	Quién lo hizo

mesa

Mesas del salón con zona, capacidad y estado libre/ocupada.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
nombre	`VARCHAR(30)`	No		único	Nombre (único)
zona	`zona_mesa`	No			interior o exterior
capacidad	`INT`	No			Lugares (> 0)
estado	`estado_mesa`	No	'libre'_mesa		libre u ocupada
estado_desde	`TIMESTAMPTZ`	No	now()		Desde cuándo está en ese estado
activo	`BOOLEAN`	No	true		FALSE = dada de baja (no cuenta para el aforo)

fichaje

Ingresos y egresos del personal (horas trabajadas).

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
id_usuario	`INT`	No		FK → usuario	Empleado
turno	`turno_trabajo`	No			produccion o servicio
ingreso	`TIMESTAMPTZ`	No			Momento de ingreso
egreso	`TIMESTAMPTZ`	Sí			Momento de egreso (vacío = jornada abierta; nunca anterior al ingreso)
nota	`VARCHAR(300)`	No	vacío		Observación
manual	`BOOLEAN`	No	false		TRUE si lo cargó o corrigió el administrador
editado_por	`INT`	Sí		FK → usuario	Quién lo corrigió
creado_en	`TIMESTAMPTZ`	No	now()		Momento del registro

pedido

Cuenta de una mesa o de la barra; se cobra o se cancela.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
canal	`canal_venta`	No			mesa o barra
id_mesa	`INT`	Sí		FK → mesa	Mesa (obligatoria en canal mesa, vacía en barra)
etiqueta	`VARCHAR(40)`	No	vacío		Nombre del cliente en barra
estado	`estado_pedido`	No	'abierto'_pedido		abierto, cerrado (cobrado) o cancelado

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id_usuario	`INT`	No		FK → usuario	Quién abrió la cuenta
creado_en	`TIMESTAMPTZ`	No	now()		Apertura
cerrado_en	`TIMESTAMPTZ`	Sí			Cobro o cancelación
total	`NUMERIC(14,2)`	Sí			Total cobrado
medio_pago	`medio_pago`	Sí			efectivo, tarjeta, mercado_pago o transferencia
id_cobrador	`INT`	Sí		FK → usuario	Quién cobró

item_pedido

Cada línea pedida, con el precio y costo «congelados» al pedir y su estado en cocina/barra.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
id_pedido	`INT`	No		FK → pedido	Cuenta
id_producto	`INT`	No		FK → producto	Producto pedido
nombre	`VARCHAR(100)`	No			Nombre al momento de pedir
destino	`destino_producto`	No			barra o cocina
cantidad	`INT`	No			Unidades (> 0)
precio_unitario	`NUMERIC(14,2)`	No			Precio al pedir (no cambia si luego sube la carta)
costo_unitario	`NUMERIC(14,2)`	Sí			Costo al pedir (para el margen)
nota	`VARCHAR(200)`	No	vacío		Aclaración («sin hielo»)
estado	`estado_item`	No	'pendiente'_item		pendiente, preparando, listo, entregado o anulado
creado_en	`TIMESTAMPTZ`	No	now()		Cuándo se pidió
preparando_en	`TIMESTAMPTZ`	Sí			Cuándo se empezó a preparar
listo_en	`TIMESTAMPTZ`	Sí			Cuándo quedó listo
entregado_en	`TIMESTAMPTZ`	Sí			Cuándo se entregó
id_usuario	`INT`	Sí		FK → usuario	Quién tomó el pedido

cierre_caja

Cierre de caja de una noche y canal, con el arqueo del efectivo.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
fecha_jornada	`DATE`	No		único (con otro campo)	Noche comercial (06:00 a 06:00)
canal	`VARCHAR(10)`	No	'todos'	único (con otro campo)	todos, mesa o barra (un cierre por noche y canal)
cerrado_en	`TIMESTAMPTZ`	No	now()		Momento del cierre
id_usuario	`INT`	No		FK → usuario	Quién cerró
cuentas	`INT`	No			Cuentas cobradas

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
ventas_total	`NUMERIC(14,2)`	No			Total vendido
ventas_efectivo	`NUMERIC(14,2)`	No			Vendido en efectivo
ventas_tarjeta	`NUMERIC(14,2)`	No			Vendido con tarjeta
ventas_mercado_pago	`NUMERIC(14,2)`	No			Vendido con Mercado Pago
ventas_transferencia	`NUMERIC(14,2)`	No			Vendido por transferencia
fondo_inicial	`NUMERIC(14,2)`	No	0		Efectivo con el que se abrió la caja
retiros	`NUMERIC(14,2)`	No	0		Retiros de efectivo (proveedores, propinas)
efectivo_esperado	`NUMERIC(14,2)`	No			Fondo + ventas en efectivo − retiros
efectivo_contado	`NUMERIC(14,2)`	No			Lo que se contó realmente
diferencia	`NUMERIC(14,2)`	No			Contado − esperado
nota	`VARCHAR(300)`	No	vacío		Comentario del cierre

auditoria

Registro de quién hizo qué, cuándo y con qué resultado. No se edita desde el sistema.

Campo	Tipo	Nulo	Por defecto	Clave	Descripción
id	`INT`	No	autoincremental	PK	Identificador
fecha_hora	`TIMESTAMPTZ`	No	now()		Momento
id_usuario	`INT`	Sí			Quién (vacío si no hubo sesión)
usuario	`VARCHAR(150)`	No	vacío		Nombre al momento (por si luego se borra)
rol	`VARCHAR(20)`	No	vacío		Rol al momento
accion	`VARCHAR(60)`	No			Código de la acción (p. ej. producto.precio)
entidad	`VARCHAR(40)`	No	vacío		Tipo de dato afectado
id_entidad	`VARCHAR(40)`	No	vacío		Identificador del dato afectado
resultado	`VARCHAR(10)`	No	'ok'		ok, denegado o fallido
detalle	`TEXT`	No	vacío		JSON con antes/después y datos útiles (sin secretos)
ip	`VARCHAR(64)`	No	vacío		Dirección IP de origen

6.3 Relaciones entre tablas

Relación (cardinalidad)	Campo(s)	Significado
proveedor 1 — N insumo	insumo.id_proveedor (opcional)	Proveedor habitual del insumo
proveedor 1 — N lote_insumo	lote_insumo.id_proveedor (opcional)	A quién se compró ese lote
producto 1 — N detalle_receta N — 1 insumo	detalle_receta.id_producto / id_insumo	Relación muchos a muchos Producto–Insumo (la receta); par único
insumo 1 — N presentacion_insumo	presentacion_insumo.id_insumo	Envases de un insumo
insumo 1 — N lote_insumo	lote_insumo.id_insumo	Cada compra o ajuste es un lote
insumo 1 — N movimiento_stock	movimiento_stock.id_insumo	Historial de movimientos del insumo
lote_insumo 1 — N movimiento_stock	movimiento_stock.id_lote (opcional)	De qué lote salió o a cuál entró

Relación (cardinalidad)	Campo(s)	Significado
mesa 1 — N pedido	pedido.id_mesa (opcional: nulo en la barra)	Cuentas de la mesa; solo una abierta a la vez (índice único parcial)
pedido 1 — N item_pedido	item_pedido.id_pedido	Líneas de la cuenta
producto 1 — N item_pedido	item_pedido.id_producto	Qué producto se pidió (con precio y costo congelados)
usuario 1 — N sesion	sesion.id_usuario (borrado en cascada)	Sesiones abiertas
usuario 1 — N fichaje	fichaje.id_usuario · editado_por	Horas del empleado y quién las corrigió; una sola jornada abierta por persona
usuario 1 — N pedido / item_pedido	pedido.id_usuario · id_cobrador · item_pedido.id_usuario	Quién abrió, cobró y tomó cada pedido
usuario 1 — N cierre_caja · lote_insumo · movimiento_stock	id_usuario	Responsable de cada registro

auditoria guarda el usuario por nombre y por id sin clave foránea a propósito: el registro de auditoría debe sobrevivir aunque el usuario cambie o se elimine.

6.4 Tipos enumerados, restricciones e índices

Tipo	Valores
rol_usuario	administrador, cajero, barra, camarero, host, cocina
destino_producto	barra, cocina
zona_mesa	interior, exterior
estado_mesa	libre, ocupada
turno_trabajo	produccion, servicio
tipo_movimiento	ingreso, venta, merma, ajuste, reversion
motivo_merma	vencido, devolucion, se_cayo, consumo_interno
canal_venta	mesa, barra
estado_pedido	abierto, cerrado, cancelado
estado_item	pendiente, preparando, listo, entregado, anulado
medio_pago	efectivo, tarjeta, mercado_pago, transferencia

Garantía en la base	Cómo se implementa
Sin duplicados	`UNIQUE` en nombre de producto, insumo, proveedor, mesa y en email de usuario; (insumo, nombre) en presentaciones; (producto, insumo) en recetas; (fecha_jornada, canal) en cierres.
Valores válidos	`CHECK`: precio ≥ 0, cantidades > 0, capacidad > 0, stock máximo ≥ mínimo, egreso ≥ ingreso, canal «mesa» exige mesa y «barra» no, resultado de auditoría ∈ {ok, denegado, fallido}.
Una sola cuenta abierta por mesa	Índice único parcial `pedido_mesa_abierto` (id_mesa) donde estado = abierto.
Una sola jornada abierta por empleado	Índice único parcial `fichaje_abierto_unico` (id_usuario) donde egreso es nulo.
Integridad referencial	Claves foráneas en todas las relaciones; las sesiones se borran en cascada con el usuario.
Consultas rápidas	Índices por fecha, estado y destino en pedidos, movimientos, auditoría y cola de cocina.

7 Configuración e instalación (referencia técnica)

Los pasos para instalar y ejecutar están en el capítulo 2. Este apartado reúne las versiones, dependencias y la configuración para quien mantenga el proyecto.

7.1 Versiones y dependencias

Tipo	Paquete	Versión	Uso
Ejecución	`pg`	^8.12.0	Driver de PostgreSQL
Desarrollo	`@electric-sql/pglite`	^0.3.0	PostgreSQL embebido (sin DATABASE_URL)
Desarrollo	`react`, `react-dom`	^19.2.0	Interfaz
Desarrollo	`esbuild`	^0.27.0	Compilar la interfaz
Entorno	Node.js	≥ 22.5	Campo `engines` de `package.json`; archivo `.node-version` fija 22.22.0 para Render

7.2 Comandos del proyecto

Comando	Qué hace
`npm install`	Instala las dependencias
`npm run seed`	Carga datos de ejemplo (bloqueado en producción)
`npm run dev`	Compila la interfaz y arranca el servidor con recarga automática
`npm run build`	Compila la interfaz a `public/` (archivos con hash para caché)
`npm start`	Arranca el servidor (producción o local)
`npm test`	Ejecuta las 187 pruebas automáticas
`npm run respaldo -- crear|listar|verificar|restaurar`	Copias de seguridad por terminal

7.3 Variables de entorno

Todas son opcionales en desarrollo (ver `.env.example`). Se leen en un solo lugar: `src/config.js`.

Variable	Por defecto	Qué hace
`PORT`	3000	Puerto del servidor
`DATABASE_URL`	(vacía)	Cadena de conexión a PostgreSQL. Vacía = PGlite local. **Obligatoria en producción.**
`DB_POOL_MAX`	5	Conexiones simultáneas al motor
`NODE_ENV`	—	`production` activa los controles de producción

Variable	Por defecto	Qué hace
`ADMIN_NOMBRE / ADMIN_EMAIL / ADMIN_PASSWORD`	—	Primer administrador (se crea solo al arrancar si no hay ninguno)
`ZONA_HORARIA`	America/Argentina/Buenos_Aires	Zona para todas las fechas
`JORNADA_INICIO_HORA`	6	La «noche» comercial va de esta hora a la misma del día siguiente
`SESION_HORAS`	14	Duración de una sesión
`TRUST_PROXY`	0	Cantidad de proxies delante (afecta el límite de intentos de ingreso)
`PGLITE_DIR`	.data/pglite	Dónde guarda sus datos PGlite (`memory` = no persiste)
`RESPALDO_DIR / _CADA_HORAS / _CONSERVAR / _COMANDO_EXTERNO`	.data/respaldos · 12 · 40 · vacío	Copias automáticas (ver `docs/COPIAS.md`)

7.4 Crear o restaurar la base de datos

- **Automático:** al arrancar, el servidor ejecuta `sql/schema.sql` y crea las tablas que falten. No hace falta ningún paso manual.
- **Scripts SQL:** `sql/schema.sql` (estructura) y `sql/datos-ejemplo.sql` (datos de demostración) para usar con `psql` (capítulo 2.3).
- **Copias:** archivos `.json.gz` independientes del motor, creados desde la aplicación o con `scripts/respaldo.js`; la restauración ocurre en una sola transacción (o queda todo o no cambia nada).

7.5 Credenciales de prueba

Usuarios de ejemplo del capítulo 1 (clave `radas1234`). En producción no se cargan datos de ejemplo: el primer administrador se define con `ADMIN_EMAIL` y `ADMIN_PASSWORD` .

8 Componentes principales

Cada componente indica su archivo, su responsabilidad y las funciones clave. Los de `src/domain/` son puros (reciben datos, devuelven datos) y tienen pruebas unitarias.

8.1 Núcleo de la API

Componente	Archivo	Responsabilidad y métodos clave
Servidor HTTP	`src/http/servidor.js`	Atiende `/api/*`, la carta pública (`/carta`), los archivos de la interfaz con gzip y `ETag` (responde 304 si no cambió) y agrega cabeceras de seguridad y CSP. Limita el cuerpo de las peticiones a 100 KB.
Despachador	`src/http/despachar.js` · `crearDespachador(tabla, {auditar, instantaneas})`	Para cada petición: (1) busca la ruta (404/405 si no existe), (2) exige sesión (401), (3) verifica que el rol esté permitido (403), (4) ejecuta el *handler* y (5) registra la auditoría con el «antes y después». Convierte cualquier error en una respuesta JSON uniforme.
Tabla de rutas	`src/api.js` y `src/modules/*/*.rutas.js`	74 rutas con la forma `{ metodo, ruta, roles | publico, status, handler }`. Agregar una ruta es agregar un objeto a la lista.
Validación	`src/http/validar.js` · `v.objeto, v.texto, v.numero, v.opcion, v.lista, v.booleano`, `validar(esquema, body)`, `idDe(param)`	Valida y normaliza toda entrada (recorta textos, rechaza campos desconocidos, comprueba rangos). Devuelve errores por campo en castellano.
Errores	`src/http/errores.js` · `HttpError`, `ErrorValidacion`, `respuestaDeError`	Traduce errores de negocio y códigos de PostgreSQL (23505, 23503, 23514) a estados HTTP con mensajes claros.
Acceso a datos	`src/db.js` · `db.query`, `db.transaction(fn)`	Interfaz única sobre `pg` (pool, tiempo de espera de conexión de 30 s, manejo de errores del pool) y sobre PGlite.
Autenticación	`src/modules/auth/` · `seguridad.js`, `sesiones.js`	Hash de contraseñas con scrypt; sesiones con token aleatorio almacenado como hash (`crearSesion`, `buscarSesion`); límite de 5 intentos fallidos por IP y email (429).
Roles	`src/roles.js`	Constantes de roles y grupos de permisos reutilizados por todas las rutas (`SOLO_ADMIN`, `SALON`, `PEDIDOS`, `KDS`, `CAJA`…).

8.2 Dominio (lógica pura)

Archivo	Funciones clave	Qué resuelve
`costos.js`	`costoUnitarioDeCompra`, `costoDeReceta`, `calcularMargen`, `precioParaMargen`, `aplicarPorcentaje`	Costo de una receta con el último precio de compra, margen en $ y %, precio necesario para un margen deseado y ajuste porcentual con redondeo.
`stock-fefo.js`	`asignarFefo(lotes, cantidad)`	Decide de qué lotes salen las cantidades: primero lo que vence antes («primero vence, primero sale»); lo que no vence va último; informa el faltante y evita errores de coma flotante.
`alertas.js`	`calcularAlertas`	Reponer (bajo el mínimo), sobrestock, por vencer y vencidos.

Archivo	Funciones clave	Qué resuelve
`pedidos.js`	`transicionValida`, `puedeCambiar`, `totalDe`, `nivelDeEspera`, `agruparPorProducto`	Máquina de estados de las líneas (pendiente → preparando → listo → entregado / anulado), quién puede cada cambio, total sin lo anulado y semáforo de espera en cocina y barra.
`horas.js`	`minutosTrabajados`, `esOlvidoEgreso`, `resumirHoras`	Horas por empleado, turno y día, incluidas jornadas que cruzan la medianoche.
`ventas.js`	`calcularArqueo`, `resumirVentas`, `jornadaDe`, `rangoDeJornadas`	Efectivo esperado = fondo + ventas en efectivo − retiros; resúmenes por noche, hora, medio y canal.
`informes.js`	`rentabilidad`, `compararPeriodos`, `consumoRealVsTeorico`, `valorizarStock`, `resumenMermas`, `comprasSugeridas`	Cálculos de los informes del dueño.
`presentaciones.js`	`aBase`, `desglosar`, `equivalencia`, `normalizarPresentaciones`	Conversión entre envases («3 botellas + 200 ml») y unidad base.
`fechas.js`	`fechaEn`, `inicioDeDia`, `sumarDias`	Fechas siempre en hora de Buenos Aires, sin depender de la zona del servidor.
`qr.js`	`qrMatriz`, `qrSvg`	Genera el QR de la carta en SVG, sin librerías ni servicios externos.
`auditoria.js`	`ETIQUETAS`, `diferencias`, `limpiar`, `serializarDetalle`	Etiqueta legible de cada acción, cálculo de cambios (antes → después) y borrado de secretos.

8.3 Módulos de negocio

Módulo	Responsabilidad	Funciones clave
pedidos	Cuentas por mesa y barra, cola de cocina/barra, cobro, cancelación y movimiento de cuentas.	`crear` (abre la cuenta, ocupa la mesa y descuenta stock por receta en una transacción), `agregar`, `cambiarEstado`, `anularItem`, `cerrar`, `cancelar`, `moverCuenta`, `moverItems`, `cola`
stock	Lotes, ingresos, mermas, ajustes y alertas.	`registrarIngreso` (crea el lote y actualiza el costo del insumo y de los platos que lo usan), `consumirInsumo` (FEFO), `revertirVenta` (devuelve al lote exacto), `registrarMerma`, `registrarAjuste`, `alertas`
catalogo	Productos, recetas, precios, insumos con presentaciones y proveedores.	`productos.servicio`: `crear`, `actualizar`, `reemplazarReceta`, `cambiarPrecio`, `ajustarPrecios`; `eliminarSinHistorial` (borrado real solo si no hay historial)
ventas	Jornada comercial, caja por canal, cierres y reportes.	jornada actual, `cerrar caja` (arqueo), `reabrir`, resumen por período, CSV
fichajes	Ingreso/egreso del personal y correcciones del administrador.	ingreso, cierre, resumen de horas, planilla CSV
mesas	Disponibilidad y administración de mesas.	listado con estado, cambio de estado, alta/edición/baja
usuarios · **auth**	Empleados, roles, contraseñas y sesiones.	alta, baja, cambio de rol, restablecer clave (cierra sesiones); login/logout
informes	Informes del dueño.	rentabilidad, comparar períodos, consumo, stock y mermas, compras sugeridas
respaldo	Copias de seguridad.	`crear`, `verificar`, `restaurar` (transacción única), programador cada 12 h, comando externo opcional

Módulo	Responsabilidad	Funciones clave
auditoria	Registro de actividad.	consulta con filtros, paginación y CSV; fotos «antes/después» por ruta (`instantaneas.js`)

8.4 Flujo completo de un pedido

1. **Camarero** toca «Nueva cuenta», elige la mesa y arma el pedido → `POST /api/pedidos`.
2. El despachador verifica sesión y que el rol pueda tomar pedidos; `validar` revisa mesa, productos y cantidades.
3. `pedidos.servicio.crear` abre una **transacción**: crea el `pedido` y sus `item_pedido` (con precio y costo congelados), marca la mesa como ocupada y, para cada insumo de la receta, llama a `consumirInsumo`, que usa `asignarFefo` para descontar de los lotes que vencen antes y deja un `movimiento_stock` por lote.
4. Si algo falla, la transacción se deshace: no queda ni la cuenta ni el descuento de stock a medias. Si dos personas abren cuenta en la misma mesa a la vez, el índice único parcial hace ganar a una sola y la otra recibe «Esa mesa ya tiene una cuenta abierta».
5. **Cocina o barra** ven la línea en su pantalla (consulta cada pocos segundos): «Empezar» → preparando, «Ya está» → listo. Cada cambio pasa por `transicionValida` y `puedeCambiar`.
6. **Camarero** entrega; **cajero o camarero** cobra → `POST /api/pedidos/:id/cerrar`: se cierra la cuenta, se registra el medio de pago y se libera la mesa.
7. Si se anula una línea pendiente, `revertirVenta` devuelve el stock exacto a los lotes de origen.
8. Cada operación que modifica datos deja una línea en **auditoría**.

9 Gestión de errores y validaciones

Los errores se manejan en **capas**: cada una atrapa lo que le corresponde y todas devuelven mensajes en castellano que dicen qué pasó y qué hacer. Los mensajes técnicos (trazas) quedan solo en el registro del servidor; el usuario nunca los ve.

9.1 Capas de validación

Capa	Qué controla	Cómo se ve
Formulario (interfaz)	Campos obligatorios (`required`), tipos y rangos de los inputs, números escritos a la argentina (`5.000`, `1.500,50`).	El navegador marca el campo; el botón no envía.
Validador del servidor (`validar.js`)	Cada ruta declara un esquema: obligatorio, tipo, largo, rango, email, opciones, listas sin repetidos. Rechaza lo inesperado.	HTTP 400 «Datos inválidos» con la lista `detalles` por campo («Es obligatorio», «Debe ser mayor o igual a 0»); la interfaz lo muestra debajo de cada campo.
Reglas de negocio (servicios)	Permisos finos, estados válidos, stock, cajas cerradas, mesas ocupadas, etc.	HTTP 403/404/409/422 con un mensaje específico («Todavía hay pedidos sin terminar. Esperá a que estén listos o anulalos antes de cobrar»).
Restricciones de la base	`UNIQUE`, `CHECK`, `FOREIGN KEY`, índices únicos parciales. Última defensa aunque falle una capa anterior.	Se traducen: 23505 → 409 «Ya existe un … con ese nombre»; 23503 → 409 «otros datos todavía lo usan. Dalo de baja»; 23514 → 400 «fuera de rango».

9.2 Códigos de respuesta

Código	Significado	Ejemplo de mensaje
400	Datos inválidos	El stock máximo no puede ser menor que el mínimo
401	Sin sesión o sesión vencida	La sesión no es válida o venció
403	Rol sin permiso	No tenés permiso para esta acción
404	No existe	Mesa no encontrada
409	Conflicto con el estado actual o duplicado	Ya existe un proveedor con ese nombre · La cuenta ya está cerrada
422	Referencia inválida	El proveedor no existe o está dado de baja
429	Demasiados intentos de ingreso	Demasiados intentos fallidos. Probá de nuevo en unos minutos
500	Error inesperado	Error interno del servidor (el detalle queda en el registro del servidor)

9.3 Errores de conexión y situaciones excepcionales

Situación	Qué hace el sistema
La base de datos no responde al arrancar	Reintenta varias veces (mensaje «Esperando a la base de datos (n/m)…») y, si no logra conectar, termina con «No se pudo conectar a la base de datos: …». Con `docker compose` o en Render la base puede tardar en estar lista; el reintento lo cubre.

Situación	Qué hace el sistema
Se corta una conexión del pool en pleno uso	El pool registra el aviso y no tira el servidor; la siguiente consulta abre una conexión nueva. Hay un tiempo de espera de 30 s para conectar (la base gratuita de Neon se «duerme» y despierta en 1 a 3 s).
El navegador pierde la conexión con el servidor	El cliente de la API muestra «No se pudo conectar con el servidor. Revisá la conexión y probá de nuevo.» y el formulario queda abierto para reintentar.
La sesión vence o se cierra desde otro lugar	El cliente recibe 401, borra la sesión local y vuelve a la pantalla de ingreso. Una baja o un cambio de clave cierra las sesiones de esa persona al instante.
Dos personas modifican lo mismo a la vez	Transacciones y restricciones únicas: gana una; la otra recibe «La cuenta de esa mesa cambió: probá de nuevo». Hay una prueba de pedidos simultáneos que verifica que no se pierde stock.
Se vende sin stock	Se permite (el bar sigue vendiendo) y se avisa el faltante; el siguiente ingreso lo cubre. Las anulaciones mantienen coherentes los lotes.
Archivo de copia dañado	La restauración valida el archivo primero y corre en **una transacción**: si falla, la base queda como estaba.
Intentos de ingreso repetidos	A los 5 fallos por IP y email, bloqueo temporal (429). Los fallos y los accesos denegados quedan en auditoría.
Error no previsto en el servidor	Responde 500 con un mensaje genérico, registra la traza en el log y no filtra detalles internos al usuario.

10 Pruebas implementadas (resumen)

El proyecto tiene **187 pruebas automáticas** que se ejecutan con `npm test` en unos 10 segundos, y un recorrido de navegador de 62 pasos. Pasan tanto sobre SQLite en memoria (sin instalar nada) como contra **PostgreSQL 16 real**. La evidencia de ejecución está en los capítulos 16 y 17.

Archivo	Tipo	Casos	Qué cubre
`dominio.test.js`	Unitaria	20	Fechas, horas, FEFO, alertas, costos y márgenes, estados de pedido, KDS, jornada, arqueo, números a la argentina
`presentaciones.test.js`	Unitaria	6	Envases y unidad base
`qr.test.js`	Unitaria	6	Generación del QR de la carta
`informes.test.js`	Unitaria	11	Rentabilidad, comparación, consumo, compras sugeridas
`catalogo.test.js`	Integración (API + BD)	16	Proveedores, insumos, recetas, precios, márgenes, bajas
`eliminar.test.js`	Integración	5	CRUD completo, eliminación definitiva y protegida, auditoría
`pedidos.test.js`	Integración	15	Pedido → stock por receta, KDS, estados, cobro, anulaciones, concurrencia
`caja-canales.test.js`	Integración	7	Caja por canal, cierres, reapertura
`mover.test.js`	Integración	9	Mover cuentas y productos entre mesas
`respaldo.test.js`	Integración	10	Copias: ida y vuelta, transacción, CLI, cobertura de todas las tablas
`auth.test.js`, `personal.test.js`, `stock.test.js`, `ventas.test.js`, `auditoria.test.js`, `http.test.js`, `carta-publica.test.js`, `informes-api.test.js`, `presentaciones` …	Integración	resto	Login, matriz de permisos por rol, fichajes, stock FEFO, ventas, auditoría, servidor HTTP, carta pública
`test/e2e/navegador.mjs`	Extremo a extremo (Chromium)	62 pasos	Interfaz real → API → base: todos los roles, flujos, responsive y ticket

> **Reglas del proyecto vigiladas por pruebas:** toda ruta que modifica datos tiene etiqueta de auditoría; la copia de seguridad cubre todas las tablas del esquema; la carta pública no lleva JavaScript ni enlaces al sistema; ninguna pantalla desborda horizontalmente a 768, 1024 y 390 px. Si alguien agrega una tabla o una ruta y olvida estos puntos, las pruebas fallan.

11 Manual de Usuario · Introducción y requisitos

11.1 Para qué sirve RADAS

RADAS es la herramienta de trabajo del bar: **camareros, barra, cocina, caja y dueño** la usan durante la noche para tomar pedidos, prepararlos, cobrarlos y saber cómo va el negocio. Fuera del horario de atención, el dueño la usa para cargar la carta, controlar el stock, ver las ventas y decidir qué comprar.

Público al que está dirigido: el personal del bar (no hace falta saber de computación) y la persona que lo administra. Cada persona ve solo las pantallas de su rol.

11.2 Requisitos mínimos

Qué	Requisito
Dispositivo	Tablet, celular o computadora. Está pensada para usarse **de pie, con una mano**: botones grandes y sin necesidad de zoom.
Sistema operativo	Cualquiera con un navegador moderno: Android, iOS/iPadOS, Windows, macOS o Linux. No se instala ninguna aplicación.
Navegador	Versión reciente de Chrome, Edge, Firefox o Safari.
Conexión a internet	Necesaria para usar el sistema (datos móviles o WiFi). Las pantallas consumen muy poco: se actualizan solas cada 5 a 10 segundos.
Usuario y clave	Los entrega el administrador. Cada persona tiene el suyo: no se comparten.
Impresora (opcional)	Para el ticket de la cuenta: cualquier impresora, o una térmica de 80 mm. También se puede guardar como PDF.

11.3 Cómo entrar

1. Abrir la dirección del sistema (**https://radas.onrender.com** en la versión de demostración).
2. Escribir el **email** y la **contraseña** y tocar **Ingresar**.
3. Al terminar el turno, tocar **Salir** (arriba a la derecha). La sesión se cierra sola luego de 14 horas.

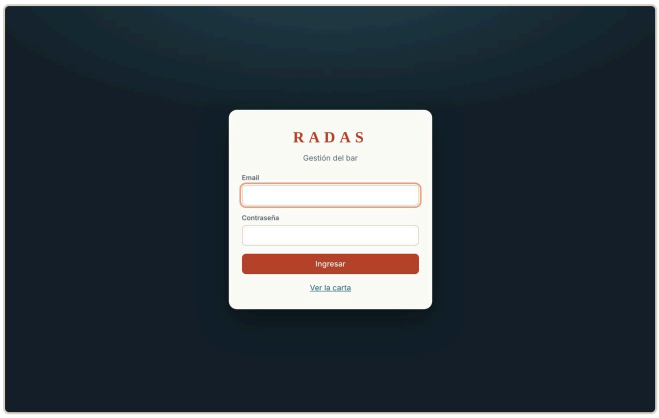


Figura 11.1. Pantalla de ingreso.

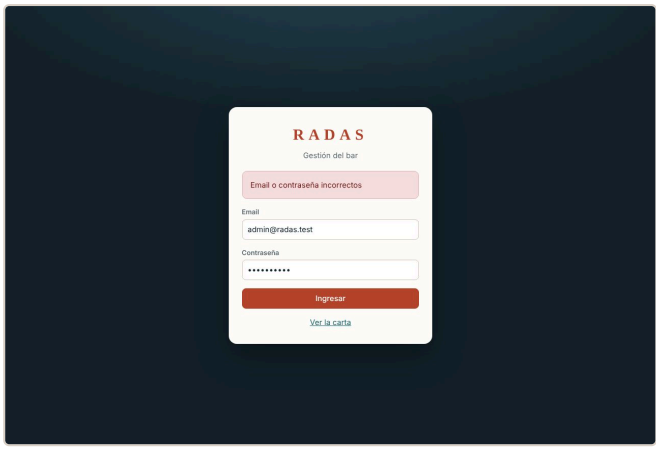


Figura 11.2. Si el email o la clave no coinciden, el sistema avisa con un mensaje claro y no dice cuál de los dos falló (por seguridad).

12 Pantalla principal y menús

Al ingresar se ve el **menú lateral** (a la izquierda; en el celular pasa a una barra horizontal arriba, que se desliza con el dedo) y el contenido de la pantalla elegida. Arriba a la derecha aparecen el nombre y el rol de quien está usando el sistema y el botón **Salir**. El menú muestra **solo lo que corresponde al rol**.

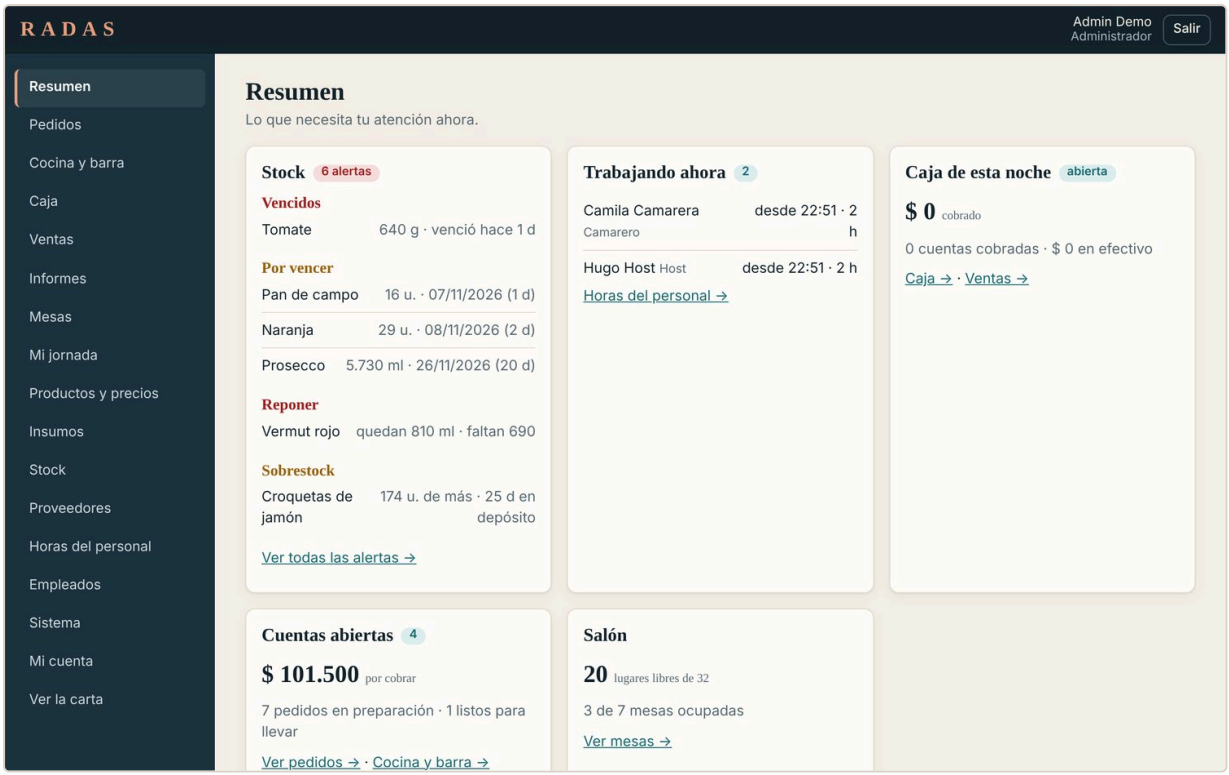


Figura 12.1. Resumen del administrador: lo que necesita atención ahora (alertas de stock, quién está trabajando, estado de la caja, cuentas abiertas y lugares libres del salón). Cada tarjeta lleva un enlace a la pantalla correspondiente.

Rol	Opciones del menú
Administrador	Resumen · Pedidos · Cocina y barra · Caja · Ventas · Informes · Mesas · Mi jornada · Productos y precios · Insumos · Stock · Proveedores · Horas del personal · Empleados · Sistema · Mi cuenta · Ver la carta
Cajero	Pedidos · Caja · Mesas · Mi jornada · Mi cuenta · Ver la carta
Camarero	Pedidos · Mesas · Mi jornada · Registrar merma · Mi cuenta · Ver la carta
Barra	Pedidos · Pantalla de barra · Mesas · Mi jornada · Registrar merma · Mi cuenta · Ver la carta
Cocina	Pantalla de cocina · Mesas · Mi jornada · Registrar merma · Mi cuenta · Ver la carta
Host	Mesas · Mi jornada · Mi cuenta · Ver la carta

12.1 Qué hace cada opción

Opción	Para qué sirve
Pedidos	Abrir cuentas, agregar productos, entregar, imprimir la cuenta, cobrar y pasar cuentas entre mesas.

Opción	Para qué sirve
Cocina y barra	Pantalla de preparación: cada pedido pasa de «En espera» a «Preparando» y a «Listo para llevar».
Mesas	Ver cuántos lugares quedan y marcar mesas ocupadas o libres (el administrador también las crea y edita).
Mi jornada	Registrar el ingreso y el egreso del turno y ver las horas trabajadas.
Registrar merma	Anotar lo que se rompió, se cayó o se venció, para que el stock coincida con la realidad.
Caja	Ver lo cobrado en la noche por medio de pago y cerrar la caja contando el efectivo.
Ventas / Informes	Reportes del dueño: ventas, ganancia, productos más rentables, comparación de períodos, consumo y qué comprar.
Productos y precios · Insumos · Stock · Proveedores	Carta, recetas, costos, mercadería, vencimientos, alertas y compras.
Horas del personal · Empleados	Quién trabajó cuánto; altas, bajas, roles y claves.
Sistema	Copias de seguridad, auditoría (quién hizo qué) y carta con QR.
Mi cuenta	Cambiar la propia contraseña.
Ver la carta	La carta pública, tal como la ven los clientes.



Figura 12.2. En el celular el mismo sistema se adapta: el menú pasa a una barra horizontal, las cuentas se apilan y los botones mantienen un tamaño cómodo para el dedo.

13 Flujos de uso paso a paso

Cada flujo indica quién lo hace, los pasos y qué se espera ver. Las capturas son del sistema real con los datos de ejemplo.

13.1 Tomar un pedido (camarero, barra, cajero)

1. Entrar a **Pedidos**. Las cuentas abiertas aparecen a la izquierda con su total y el estado de preparación («1 para llevar», «2 en preparación»).
2. Tocar **Nueva cuenta**. Elegir la pestaña **Mesa** o **Barra**. Las mesas con cuenta abierta dicen «CON CUENTA · ABRIR»; las libres, «SIN CUENTA». Tocar la mesa.
3. En la ventana del pedido, elegir la categoría (Clásicos, Coctelería de autor, Sin alcohol, Tapeo) y **tocar cada producto** para sumarlo. A la derecha se ve el pedido: con **–** y **+** se cambia la cantidad.
4. Para aclaraciones, escribir una **nota** en la línea («sin hielo, sin sal») o tocar los atajos de un toque (sin sal, sin picante, sin cebolla, sin gluten, sin lácteos, sin hielo).
5. Controlar el total abajo a la izquierda y tocar **Enviar pedido**. El pedido aparece al instante en la pantalla de cocina o de barra y el stock se descuenta solo.

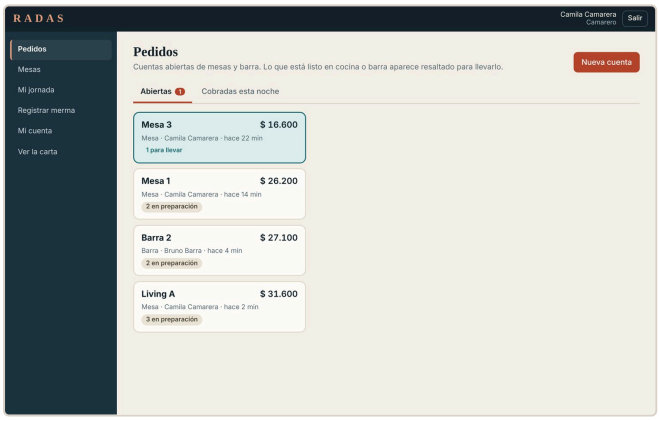


Figura 13.1. Pedidos: cuentas abiertas de mesas y barra.

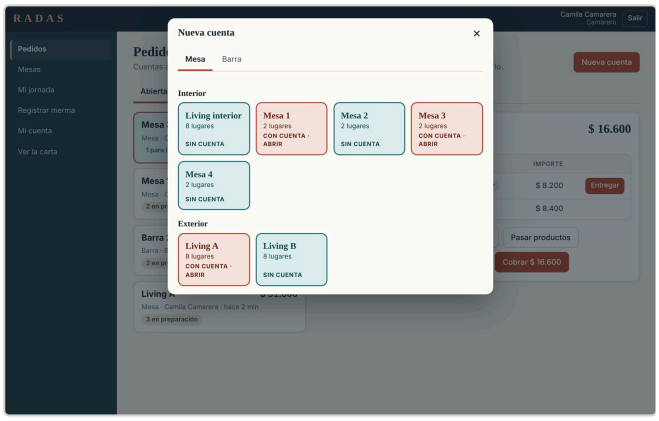


Figura 13.2. «Nueva cuenta»: se elige la mesa (rojo = ya tiene cuenta, verde = libre).

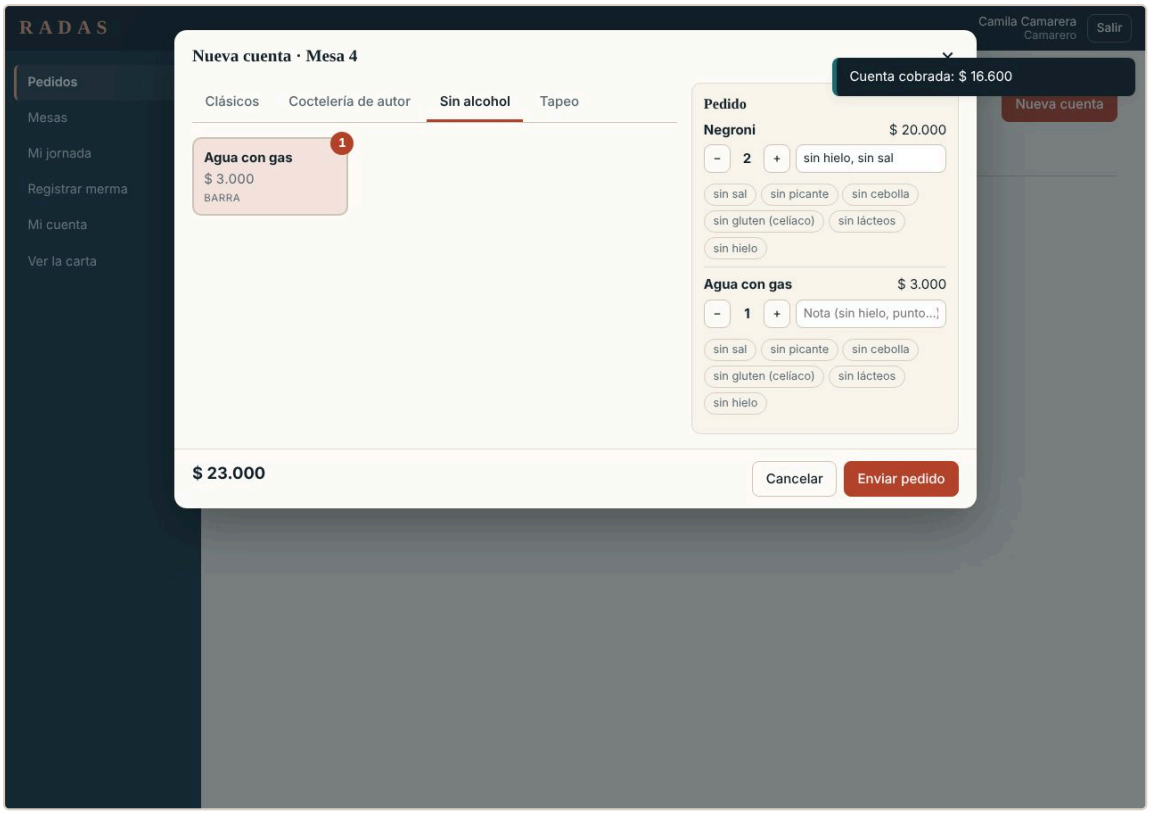


Figura 13.3. Armado del pedido de la Mesa 4: productos por categoría a la izquierda, el pedido con cantidades y notas a la derecha y el total abajo.

> Si un producto no tiene stock suficiente, **igual se puede vender** (el bar no se frena) y el sistema avisa el faltante para que el administrador lo reponga.

13.2 Preparar los pedidos: cocina y barra

1. La persona de cocina entra a **Pantalla de cocina** y la de barra a **Pantalla de barra**. Cada una ve **solo lo suyo**.
2. Hay tres columnas: **En espera → Preparando → Listo para llevar**. Cada tarjeta muestra la cantidad, el producto, la mesa y **cuánto hace que espera** (el tiempo pasa a amarillo y luego a rojo si se demora).
3. Tocar **Empezar** cuando se comienza a preparar y **Listo** (o «Ya está») cuando termina. Si se tocó por error, el botón ↩ o «Volver a preparar» lo deshace.
4. En **Agrupado** se juntan los productos iguales de cuentas distintas («3 × Negroni: Mesa 1 (2), Barra 2») para prepararlos todos juntos; en **Por cuenta** se ven como una comanda clásica.
5. Las notas importantes («sin aceite») se destacan con un cartel de advertencia. «Activar sonido» avisa cuando llega un pedido nuevo.

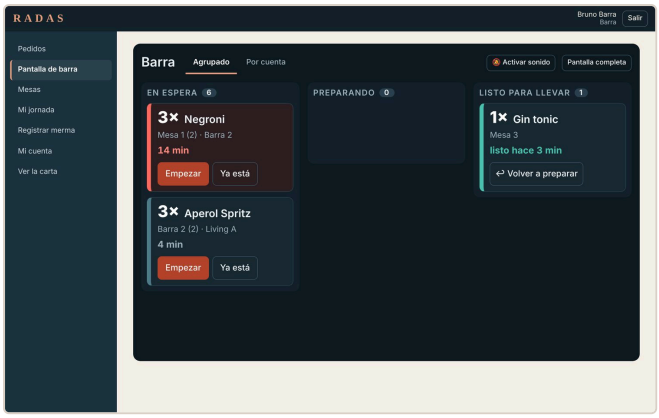


Figura 13.4. Pantalla de barra, vista agrupada.

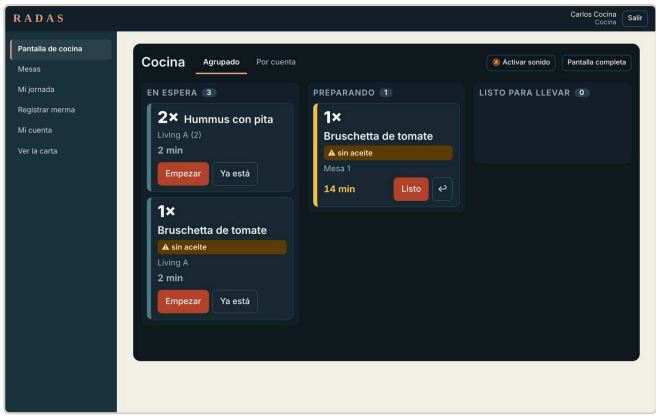


Figura 13.5. Pantalla de cocina con notas destacadas.

13.3 Entregar, imprimir la cuenta y cobrar

1. Cuando el pedido está listo, la cuenta se resalta en **Pedidos** («para llevar»). Llevarlo a la mesa y tocar **Entregar** en la línea, o **Entregar todo lo listo**.
2. Para llevar la cuenta a la mesa, tocar **Imprimir cuenta**: sale la pre-cuenta.
3. Cuando el cliente pide pagar, tocar **Cobrar $ ...**. Elegir el **medio de pago** (Efectivo, Tarjeta (POS), Mercado Pago o Transferencia) y **Confirmar cobro**. El sistema solo anota el medio: el cobro con la terminal se hace por fuera.
4. Aparece el **ticket**. Tocar **Imprimir** (o guardar como PDF) y **Cerrar**. La mesa queda libre automáticamente.
5. Para reimprimir un ticket más tarde: pestaña **Cobradas esta noche** → botón **Ticket**.

> No se puede cobrar una cuenta si todavía hay pedidos sin terminar: primero hay que esperarlos o anularlos. Si la cuenta no tiene consumos, se cancela en lugar de cobrarla.

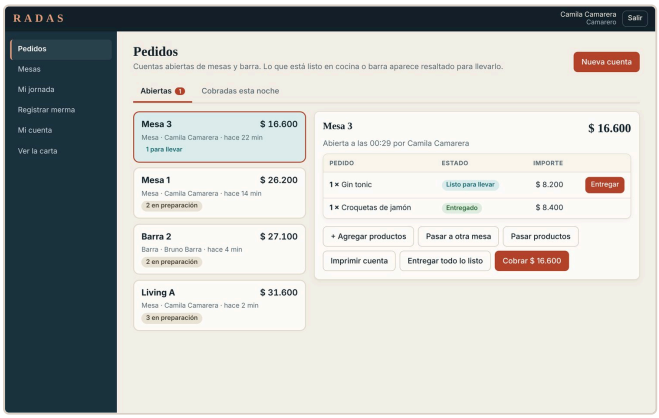


Figura 13.6. Detalle de una cuenta: estado de cada línea y botones de acción.

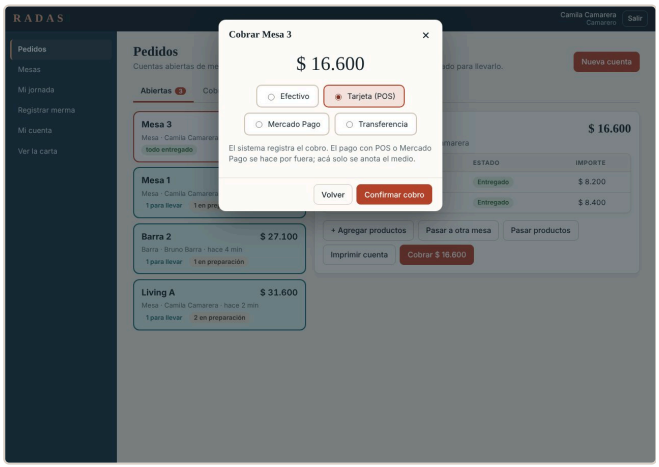


Figura 13.7. Cobro: se elige el medio de pago.

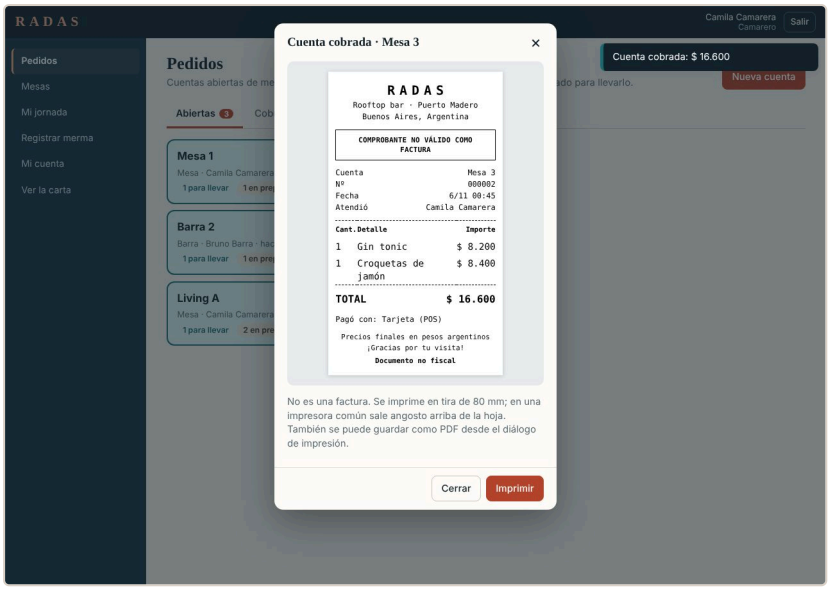


Figura 13.8. Ticket de la cuenta. Lleva el cartel «COMPROBANTE NO VÁLIDO COMO FACTURA»: es un comprobante interno, no fiscal. Sale en tira de 80 mm o se guarda como PDF.

13.4 Pasar una cuenta o productos a otra mesa

1. En **Pedidos**, abrir la cuenta y tocar **Pasar a otra mesa** (mueve toda la cuenta) o **Pasar productos** (solo lo elegido).
2. En «Pasar productos», tildar las líneas y tocar la mesa de destino. Si la mesa ya tiene una cuenta, se **suman** a esa; si no, se abre una nueva con las líneas elegidas.
3. Si el destino ya tiene cuenta al mover toda la cuenta, el sistema **pregunta si se juntan** antes de hacerlo. La mesa de origen queda libre.

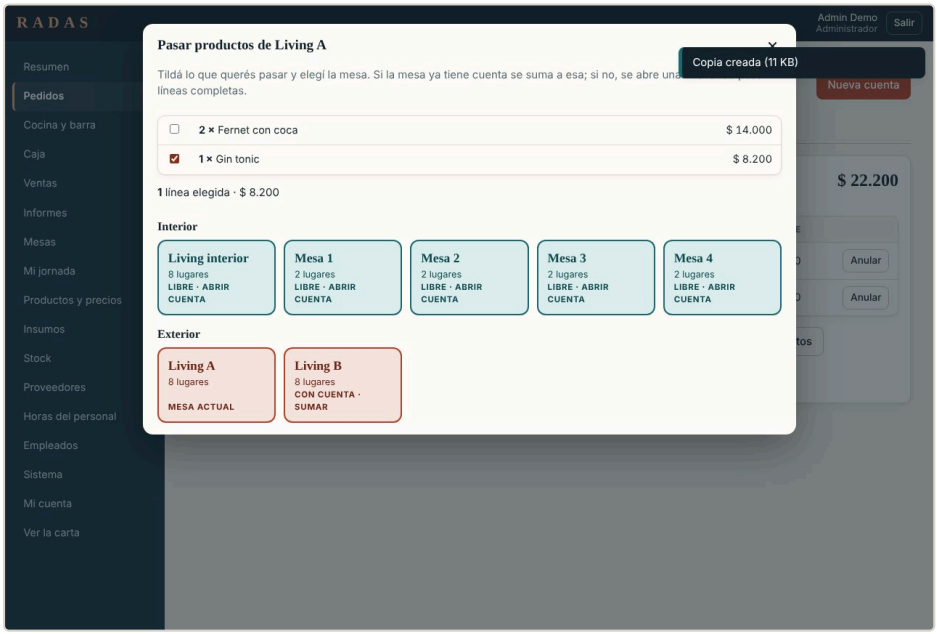


Figura 13.9. «Pasar productos»: se tildan las líneas y se elige la mesa de destino.

13.5 Ver y marcar mesas (host)

1. Entrar a **Mesas**. Arriba se ven los lugares libres (de 32), cuántos adentro y afuera, y cuántas mesas hay libres.
2. Tocar una mesa para marcarla **ocupada** o **libre**. Las mesas con cuenta abierta no se pueden liberar a mano: primero se cobra o se cancela la cuenta.
3. La pantalla se actualiza sola cada 10 segundos.

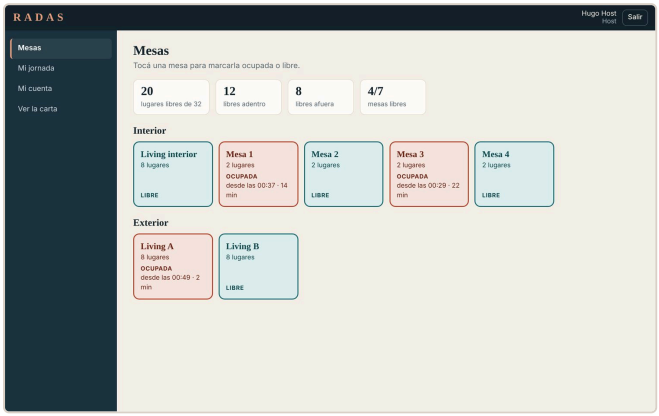


Figura 13.10. Disponibilidad de mesas (vista del host).



Figura 13.11. La misma pantalla en un celular.

13.6 Registrar el ingreso y el egreso del turno (todo el personal)

1. Al llegar, entrar a **Mi jornada**, elegir el turno (**Producción** o **Servicio**) y tocar **Registrar ingreso**.
2. Al terminar, volver a la misma pantalla y tocar **Registrar egreso**.

3. Abajo se ven las horas de los últimos días; la jornada abierta aparece como «en curso».

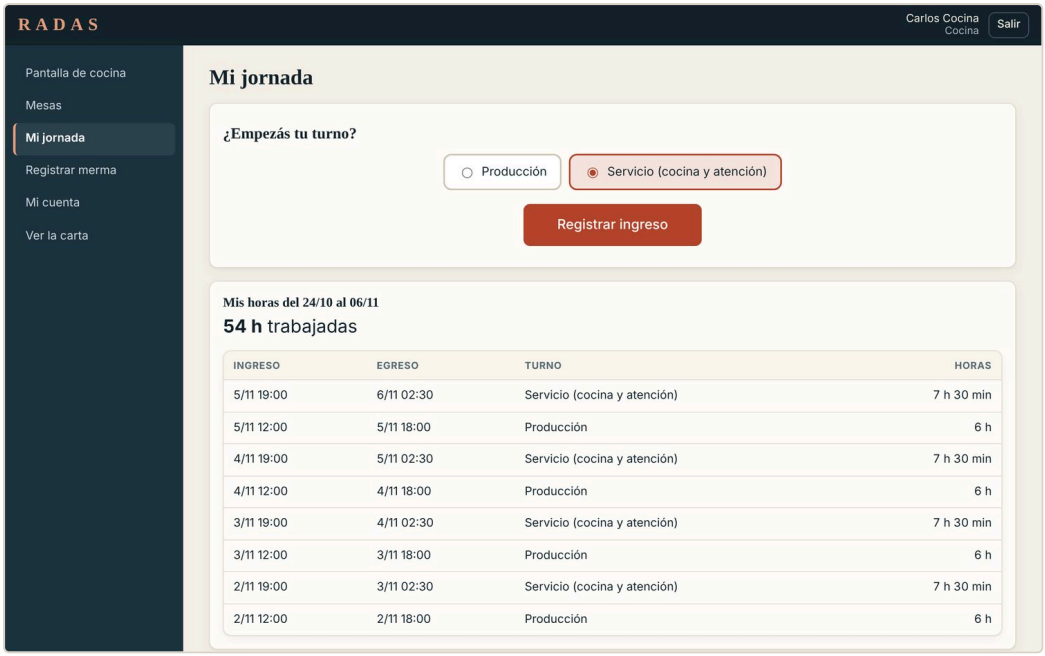


Figura 13.12. Mi jornada: elegir el turno y registrar el ingreso; abajo, las horas trabajadas.

> Si alguien olvida registrar el egreso, el administrador puede corregirlo en **Horas del personal**. Una jornada abierta por más de 16 horas se marca como olvido.

13.7 Registrar una merma

1. Entrar a **Registrar merma** (o a **Stock → Mermas** si sos administrador).
2. Elegir el **insumo**, la **medida** (envase o unidad base) y la **cantidad perdida**.
3. Elegir el **motivo** (vencido, devolución, se cayó, consumo interno), agregar una nota si hace falta y tocar el botón de guardar. El stock se descuenta de los lotes que vencen antes.

13.8 Cobrar y cerrar la caja de la noche (cajero y administrador)

1. Entrar a **Caja**. Se ve lo cobrado en la noche por medio de pago y por canal (salón y barra).
2. Elegir **«Salón y barra por separado»** (cada caja se cuenta y se cierra aparte) o **«Todo junto»**.
3. Si todavía hay **cuentas abiertas**, el sistema no deja cerrar y las lista: hay que cobrarlas o cancelarlas.
4. Cargar el **fondo de caja** con el que se abrió y los **retiros de efectivo** (pagos a proveedores, propinas). El sistema calcula el efectivo que **debería haber**.
5. Contar el efectivo y escribirlo en **Efectivo contado**. El sistema muestra al instante la **diferencia** («Faltan $ …» o «Sobran $ …»).
6. Tocar **Cerrar caja** y confirmar. Queda en «Cierres anteriores». Ya no se puede cobrar en esa caja; si hubo un error, el **administrador** puede **reabrirla**.

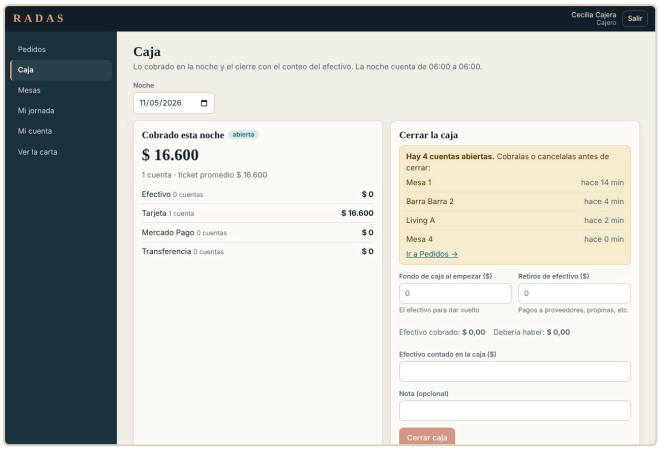


Figura 13.13. Con cuentas abiertas, la caja no se puede cerrar.

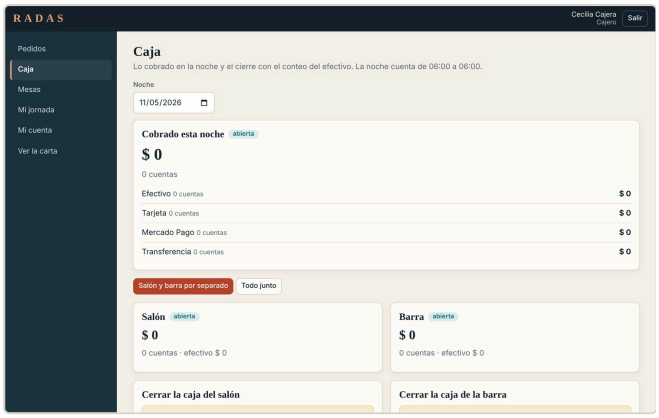


Figura 13.14. Caja: ventas por medio de pago y formulario de cierre.

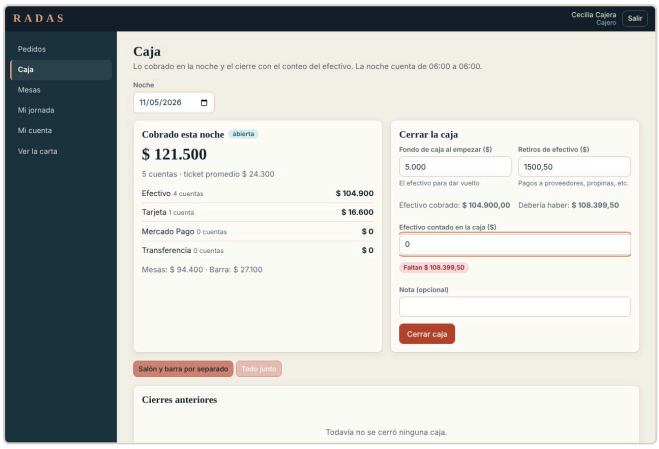


Figura 13.15. Arqueo del efectivo con la diferencia.

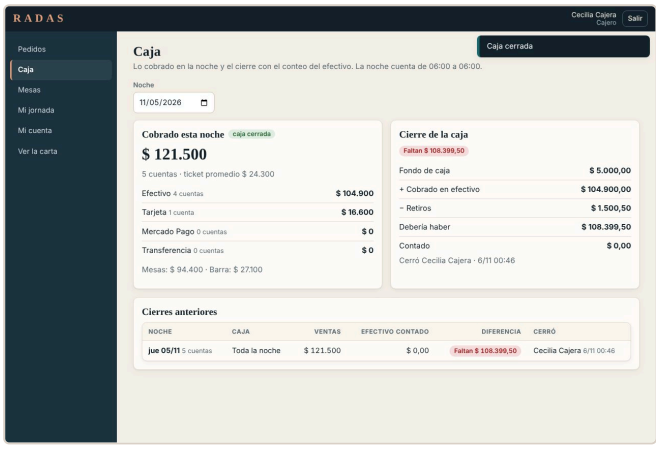


Figura 13.16. Caja cerrada: queda el historial.

13.9 Administrar la carta, los precios y las recetas (administrador)

1. **Productos y precios** muestra cada producto con su **costo** (calculado con la receta y el último precio de compra), el **precio de venta** y el **margen**.
2. Para cambiar un precio, escribir el nuevo valor directamente en la tabla: el margen se recalcula al escribir. Al terminar, tocar **Guardar cambios** y confirmar (se guardan todos juntos o ninguno).
3. Para subir o bajar muchos precios: **Ajustar precios en bloque**, por **porcentaje** o por **margen deseado**. Muestra una **vista previa** (precio actual → nuevo) y no cambia nada hasta confirmar.
4. Para crear un producto: **Nuevo producto**, completar nombre, categoría, destino (barra o cocina), descripción, precio y la **receta** (qué insumos y cuánto de cada uno). Para editar uno existente: **Editar**.
5. Para sacar un producto de la carta sin perder su historial: **Dar de baja**. Si nunca se vendió, también se puede **Eliminar definitivamente** (pide confirmación).

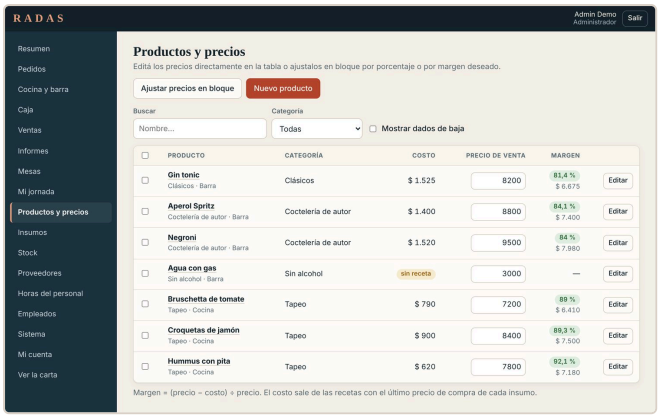


Figura 13.17. Productos y precios con costo y margen.

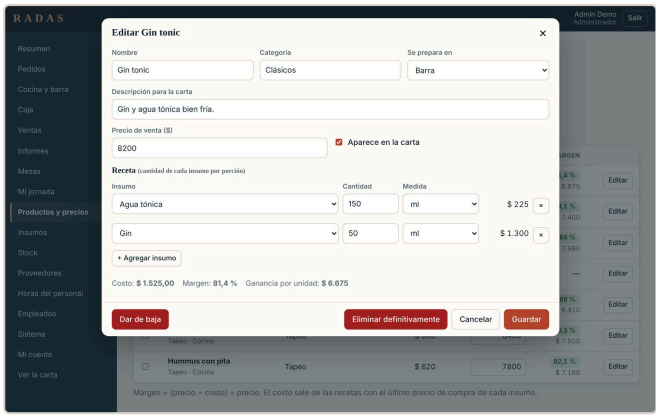


Figura 13.18. Edición de un producto con su receta; el costo y el margen se calculan al instante.

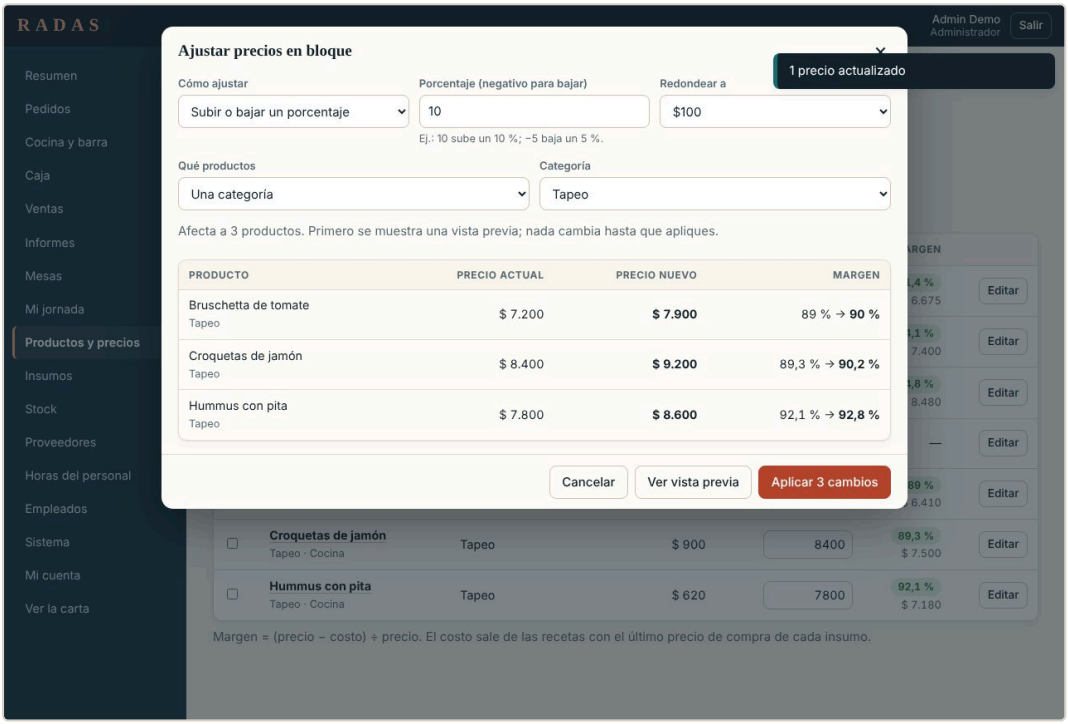


Figura 13.19. Ajuste en bloque con vista previa antes de aplicar.

13.10 Cargar mercadería y controlar el stock (administrador)

1. **Stock → Ingresar mercadería**: elegir el insumo, cómo se carga la cantidad (en envases o en unidad base), la cantidad, el **precio** (total de la compra o por unidad), el proveedor y la **fecha de vencimiento**. Tocar **Registrar ingreso**: se crea un lote y el costo de los platos que usan ese insumo se actualiza solo.
2. **Stock → Alertas** avisa: **Vencidos** (retirar y registrar la merma), **Por vencer** (usar primero), **Reponer** (bajo el mínimo) y **Sobrestock** (capital parado).
3. **Conteo físico** sirve para corregir diferencias entre lo que dice el sistema y lo que realmente hay; **Lotes** y **Movimientos** muestran el detalle.
4. **Insumos** es el catálogo de ingredientes (con mínimos, máximos, proveedor y envases como «Botella 750 ml»); **Proveedores**, la agenda de a quién se le compra.

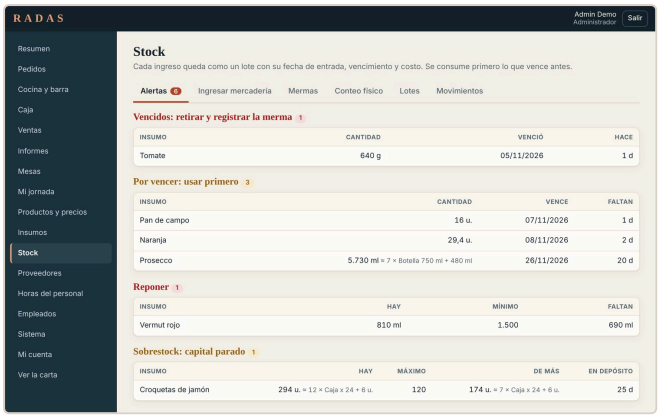


Figura 13.20. Alertas de stock.

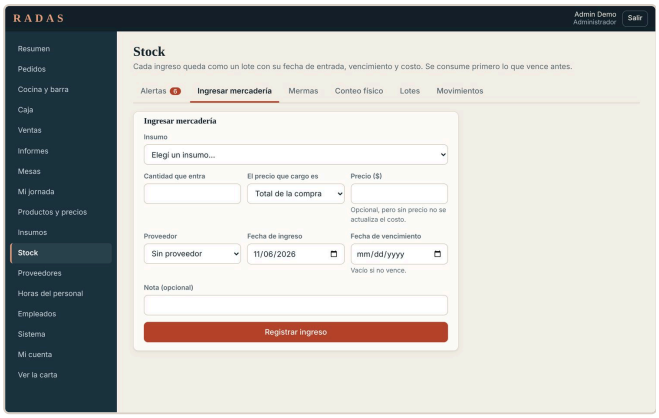


Figura 13.21. Ingreso de mercadería.

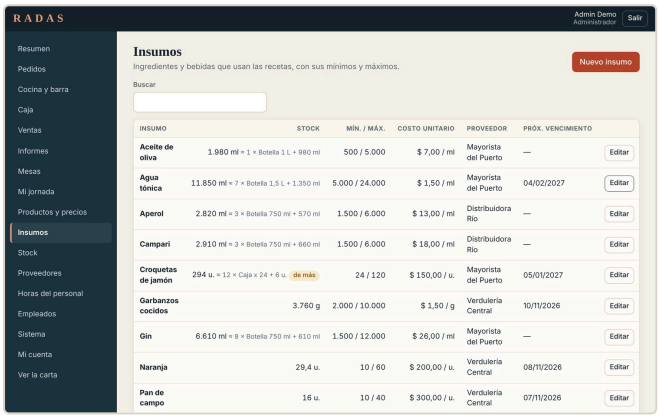


Figura 13.22. Insumos.

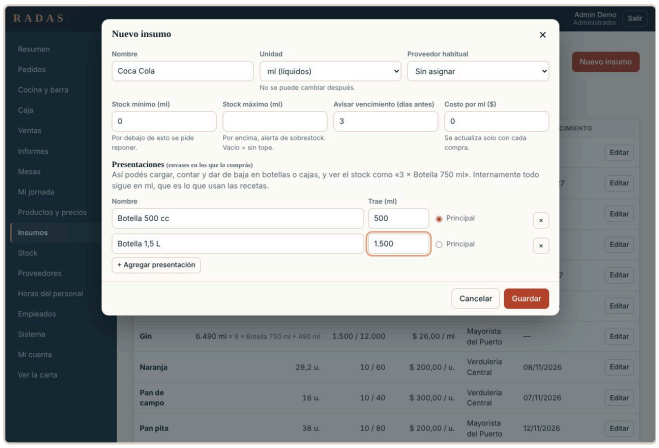


Figura 13.23. Insumo con envases («Botella 500 cc», «Botella 1,5 L»).

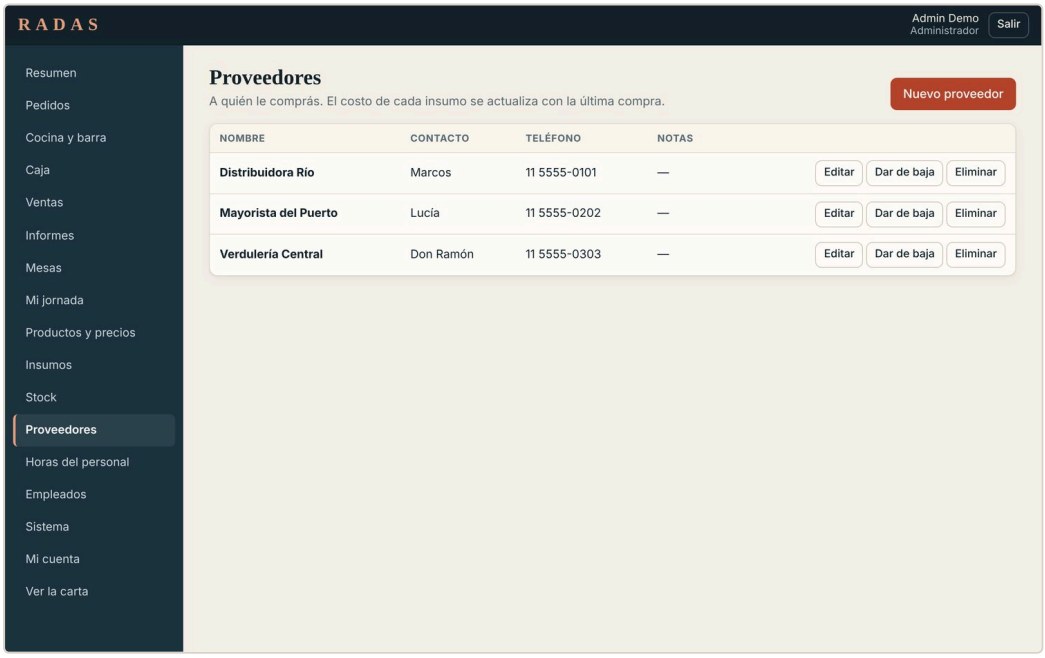


Figura 13.24. Proveedores: alta, edición, baja y eliminación con confirmación.

13.11 Empleados, horas y claves (administrador)

1. **Empleados → Nuevo empleado**: nombre, email, rol y contraseña inicial. Se puede **Editar / rol**, **Cambiar contraseña** y **Dar de baja** (o reactivar). Los cambios rigen al instante: una baja o un cambio de clave cierra las sesiones abiertas de esa persona.
2. **Horas del personal**: pestañas **Ahora** (quién está trabajando), **Horas por empleado** (por período, con detalle por día y descarga de planilla CSV) y **Jornadas** (para cargar o corregir una jornada a mano).

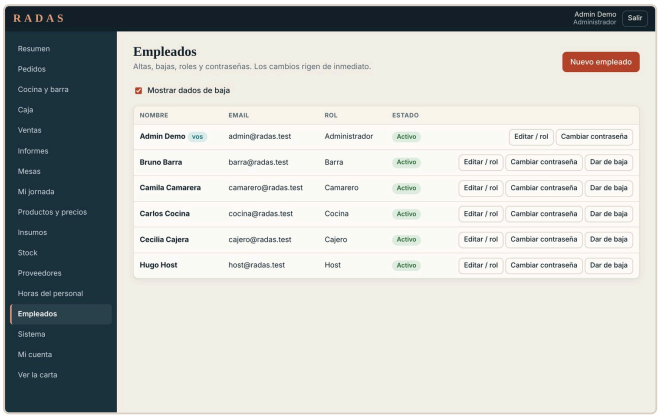


Figura 13.25. Empleados.

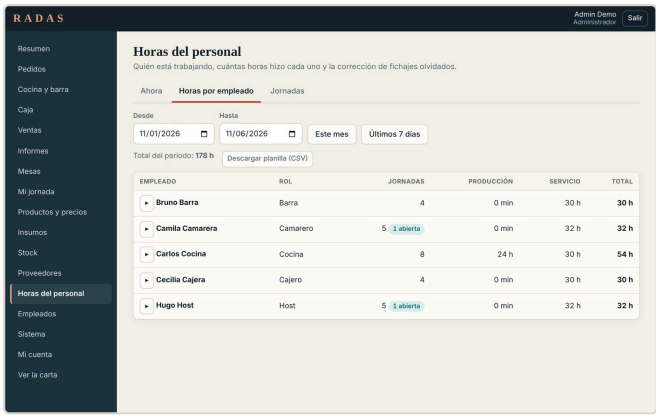


Figura 13.26. Horas por empleado con detalle por día.

13.12 Ver ventas e informes (administrador)

1. **Ventas**: elegir el período (*Esta noche, Anoche, Últimas 7, Últimas 30, Este mes* o fechas a mano). Se ven ventas, cuentas cobradas, ticket promedio, margen, ventas por noche, pedidos por hora, medios de pago, canal y productos. La «noche» va de 06:00 a 06:00.
2. **Informes**: **Rentabilidad** (qué productos aportan más ganancia: estrellas, caballitos de batalla…), **Comparar períodos**, **Consumo real vs teórico**, **Stock y mermas** y **Qué comprar** (lista por proveedor, lista para copiar y mandar por WhatsApp).

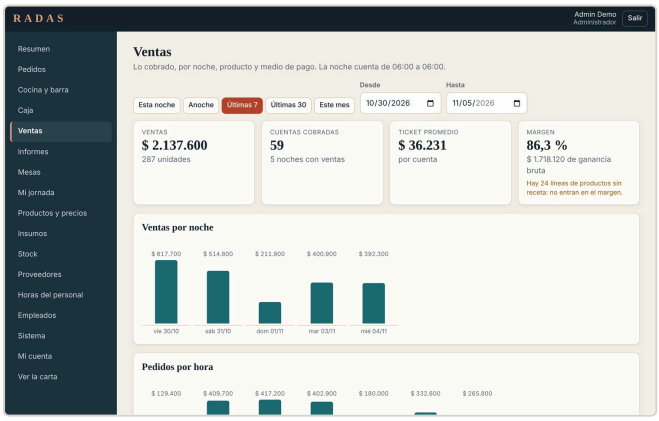


Figura 13.27. Ventas del período.

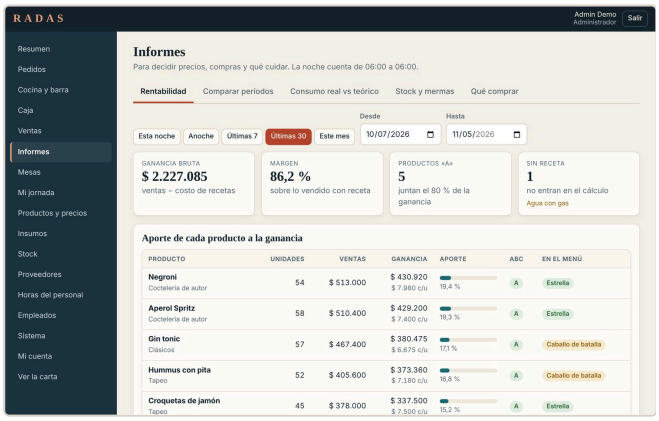


Figura 13.28. Informe de rentabilidad.

13.13 Copias de seguridad, auditoría y carta con QR (administrador)

1. **Sistema → Copias de seguridad**: el sistema hace una copia cada 12 horas. Se puede **Crear una copia ahora** y **Descargar** cualquiera para guardarla fuera del servidor (por ejemplo en el Drive). Una copia que está solo en el mismo servidor no es una verdadera copia de seguridad.

2. **Sistema → Auditoría**: quién hizo qué y cuándo (ingresos, cambios de precios con el valor anterior y el nuevo, altas, bajas, cierres…). Se filtra por fecha, persona, acción y resultado, y se descarga en CSV. No se puede modificar.
3. **Sistema → Carta con QR**: escribir la dirección pública del sistema, tocar **Imprimir cartelitos** y pegarlos en las mesas. El cliente escanea el QR y ve la carta con los precios actuales, sin usuario ni clave.

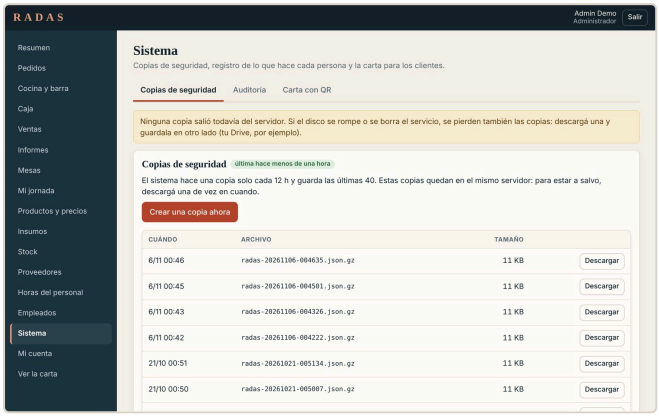


Figura 13.29. Copias de seguridad.

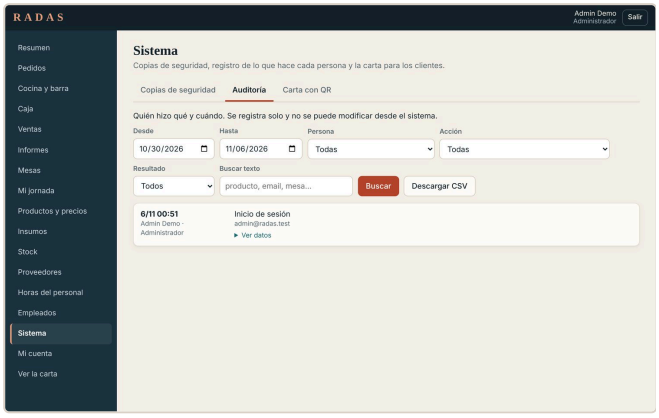


Figura 13.30. Auditoría.

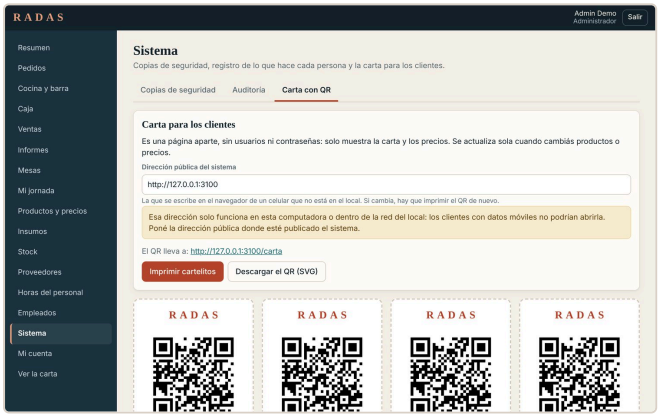


Figura 13.31. Carta con QR para imprimir.

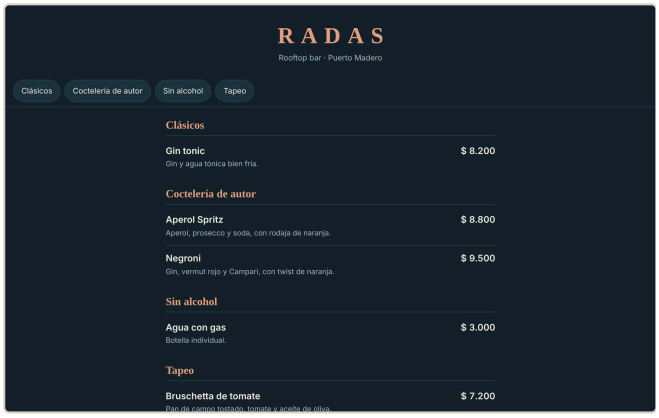


Figura 13.32. La carta que ve el cliente al escanear el QR.

13.14 Cambiar mi contraseña

1. Entrar a **Mi cuenta**.
2. Escribir la contraseña actual y la nueva (mínimo 8 caracteres) y guardar.
3. Si la actual no coincide, el sistema avisa «La contraseña actual no es correcta».

14 Mensajes y alertas frecuentes

Son mensajes reales del sistema, tal como aparecen en pantalla.

Mensaje	Qué significa	Qué hacer
Email o contraseña incorrectos	Los datos de ingreso no coinciden.	Revisar mayúsculas y espacios. Si se olvidó la clave, pedirle al administrador que la restablezca.
Demasiados intentos fallidos. Probá de nuevo en unos minutos	Hubo varios ingresos incorrectos seguidos.	Esperar unos minutos y volver a intentar con los datos correctos.
La sesión no es válida o venció	Pasaron muchas horas o el administrador cerró la sesión.	Volver a ingresar.
No tenés permiso para esta acción	Tu rol no puede hacer eso.	Pedirle a quien corresponda (por ejemplo, anular un pedido ya preparado lo hace el administrador).
No se pudo conectar con el servidor. Revisá la conexión y probá de nuevo.	Se cortó internet o el servidor se está despertando.	Verificar el WiFi o los datos y reintentar. En la versión de demostración, esperar cerca de un minuto.
Es obligatorio	Falta completar un campo marcado como obligatorio.	Completar el campo indicado.
Ya existe un producto / insumo / proveedor / mesa / usuario con ese nombre (o email)	Los nombres no pueden repetirse.	Usar otro nombre o editar el que ya existe.
Esa mesa ya tiene una cuenta abierta	Otra persona abrió una cuenta en esa mesa.	Agregar productos a la cuenta existente.
Todavía hay pedidos sin terminar. Esperá a que estén listos o anulalos antes de cobrar	Hay líneas pendientes en cocina o barra.	Esperar a que estén listas o anularlas.
La cuenta no tiene consumos: cancelala en lugar de cobrarla	Se intentó cobrar una cuenta vacía.	Usar «Cancelar cuenta».
Ya se empezó a preparar: pedile al administrador que la anule	No se puede anular algo que cocina ya empezó.	Avisar al administrador.
La cuenta de esa mesa cambió: probá de nuevo	Otra persona modificó la cuenta al mismo tiempo.	Actualizar la pantalla y repetir.
Hay N cuentas abiertas… (al cerrar caja)	No se puede cerrar la caja con cuentas sin cobrar.	Cobrarlas o cancelarlas y volver a cerrar.
La caja ya está cerrada	Ya se cerró la caja de ese canal.	Si fue un error, el administrador puede reabrirla.
Faltan $ … / Sobran $ … (cierre de caja)	El efectivo contado no coincide con el esperado.	Recontar; si la diferencia es real, anotarla en la nota del cierre.
Ya tenés una jornada abierta. Registrá primero tu egreso	Se intentó registrar un ingreso sin haber cerrado la jornada anterior.	Registrar el egreso (o pedirle al administrador que lo corrija).
El insumo se usa en recetas de productos activos	No se puede dar de baja un ingrediente que algún plato usa.	Sacarlo de las recetas o dar de baja esos productos primero.
No se puede eliminar el proveedor «…»: 3 insumos lo tienen como proveedor habitual. Para dejar de usarlo, dalo de baja…	Eliminar definitivamente solo se permite si nada usa ese dato.	Usar **Dar de baja**: queda en el historial y se puede reactivar.
Sin copias · Ninguna copia salió todavía del servidor (Sistema)	No hay copias recientes o todas están en el mismo servidor.	Crear una copia y descargarla a un lugar seguro.

15 Base de datos y CRUD

La base es **PostgreSQL** (15 tablas, descritas con su DER en el capítulo 6). Las cuatro operaciones de persistencia se hacen desde la propia aplicación, con validaciones, mensajes claros y confirmación antes de eliminar.

15.1 Cobertura de las operaciones CRUD

Entidad	Create	Read	Update	Delete
Productos	Nuevo producto (`POST /productos`)	Lista y detalle	Editar, precio, receta (`PATCH`, `PUT`)	Dar de baja; **Eliminar definitivamente** si nunca se vendió
Insumos	Nuevo insumo	Lista con stock y alertas	Editar, envases, activo	Dar de baja; **Eliminar definitivamente** si no está en recetas ni tiene stock/compras
Proveedores	Nuevo proveedor	Lista	Editar	Dar de baja; **Eliminar definitivamente** si nada lo usa
Mesas	Nueva mesa	Tablero y lista	Editar, estado	Dar de baja; **Eliminar definitivamente** si nunca tuvo pedidos
Empleados (usuarios)	Nuevo empleado	Lista	Editar rol, cambiar contraseña	Dar de baja (se conserva: tiene ventas, fichajes y auditoría a su nombre)
Pedidos y líneas	Nueva cuenta, agregar productos	Cuentas abiertas y cobradas	Cambiar estado, mover, cobrar	Anular línea, cancelar cuenta
Fichajes	Registrar ingreso / jornada manual	Mis horas, horas del personal	Registrar egreso, corregir	—
Cierres de caja	Cerrar caja	Historial de cierres	—	Reabrir (borra el cierre)

> **Por qué «dar de baja» y «eliminar definitivamente».** Un producto, insumo, proveedor o mesa que ya tiene ventas, stock o compras no se puede borrar sin dejar huérfanos los informes, la caja y la auditoría. Por eso, por defecto se **da de baja** (deja de aparecer y se puede reactivar). El **borrado real** está disponible solo cuando el registro **no tiene historial** (por ejemplo, algo cargado por error). El sistema lo verifica y, si hay historial, explica por qué no se puede y sugiere la baja.

15.2 Validaciones implementadas

Requisito	Cómo se cumple	Mensaje que ve el usuario
Evitar duplicados	Restricción `UNIQUE` en la base + traducción del error a 409.	«Ya existe un proveedor con ese nombre» (igual para producto, insumo, mesa y email de usuario)
Campos obligatorios	Atributo `required` en el formulario y esquema `validar` en el servidor (por si se salta la interfaz).	«Es obligatorio» debajo del campo

Requisito	Cómo se cumple	Mensaje que ve el usuario
Rangos y formatos	Esquemas de validación + `CHECK` en la base.	«Debe ser mayor o igual a 0», «Debe ser un email válido», «El stock máximo no puede ser menor que el mínimo»
Confirmación antes de eliminar	Ventana `Confirmar` con el nombre del registro y botón rojo; «Cancelar» no hace nada.	«El proveedor se borra para siempre. Solo se puede si ningún insumo ni compra lo usa; si no, dalo de baja.»
Integridad al eliminar	Conteo de dependencias dentro de una transacción + claves foráneas como red de seguridad.	«No se puede eliminar el proveedor «Distribuidora Río»: 3 insumos lo tienen como proveedor habitual. Para dejar de usarlo, dalo de baja...»
Mensajes claros	Todo error se traduce a castellano con la causa y qué hacer; avisos verdes de éxito («Proveedor guardado», «Proveedor eliminado»).	—
Permisos	Solo el administrador crea, edita o elimina datos del catálogo (403 para el resto).	«No tenés permiso para esta acción»
Trazabilidad	Cada alta, cambio o eliminación queda en Auditoría con el valor anterior.	—

15.3 Evidencia 1 · CRUD desde la pantalla (Proveedores)

Secuencia real capturada con el navegador automatizado (Chromium) contra el sistema funcionando; es el paso a paso del archivo `test/e2e/navegador.mjs`.

Paso	Acción	Resultado esperado y obtenido
1 · **Create**	Nuevo proveedor → Nombre «Proveedor de ejemplo CRUD», Teléfono «11 4000-0000» → Guardar	El proveedor aparece en la lista y un aviso confirma «Proveedor guardado». ✔ (Fig. 15.1 y 15.2)
2 · **Duplicado**	Nuevo proveedor con el mismo nombre → Guardar	Se rechaza con «Ya existe un proveedor con ese nombre» y no se crea nada. ✔ (Fig. 15.3)
3 · **Update**	Editar → Contacto «Laura (ventas)» → Guardar	La lista muestra el cambio. ✔ (Fig. 15.4)
4 · **Delete** (cancelar)	Eliminar → ventana de confirmación → Cancelar	No se borra nada: la fila sigue en la lista. ✔ (Fig. 15.5)
5 · **Delete** (confirmar)	Eliminar → Eliminar definitivamente	La fila desaparece y un aviso confirma «Proveedor eliminado». ✔ (Fig. 15.6)
6 · **Integridad**	Eliminar un proveedor que tiene insumos asociados	Se rechaza con un mensaje que explica el motivo y sugiere dar de baja. ✔ (Fig. 15.7)

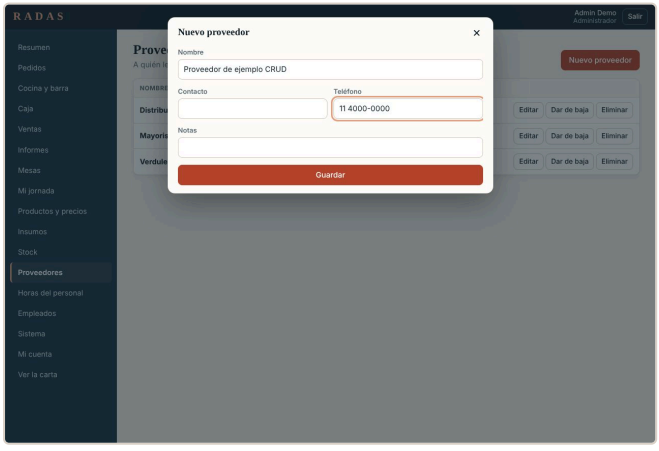


Figura 15.1. Create: formulario «Nuevo proveedor».

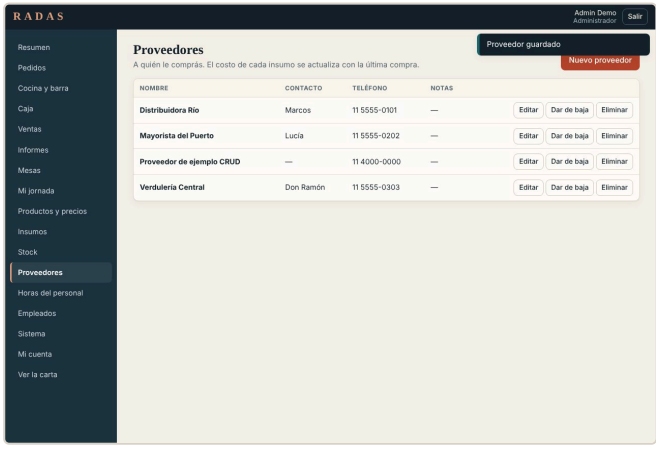


Figura 15.2. Read: el proveedor nuevo en la lista.

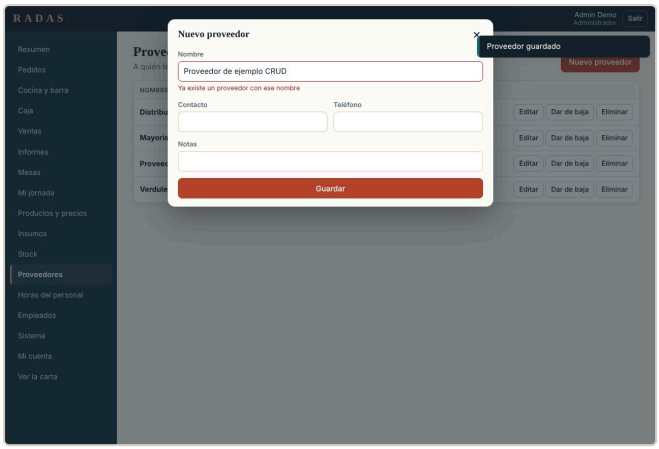


Figura 15.3. Duplicado rechazado, con mensaje bajo el campo.

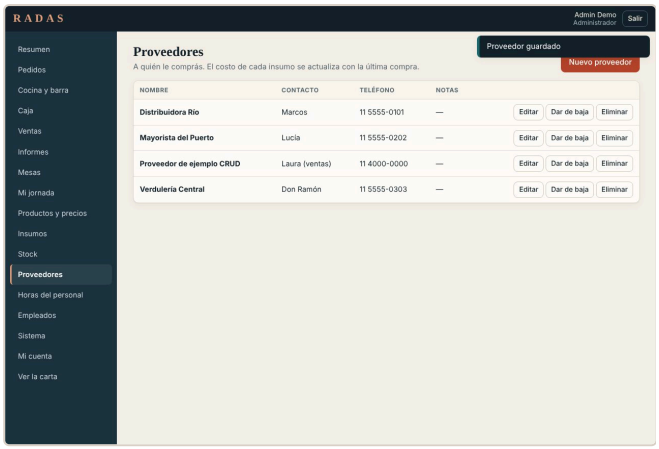


Figura 15.4. Update: contacto modificado.

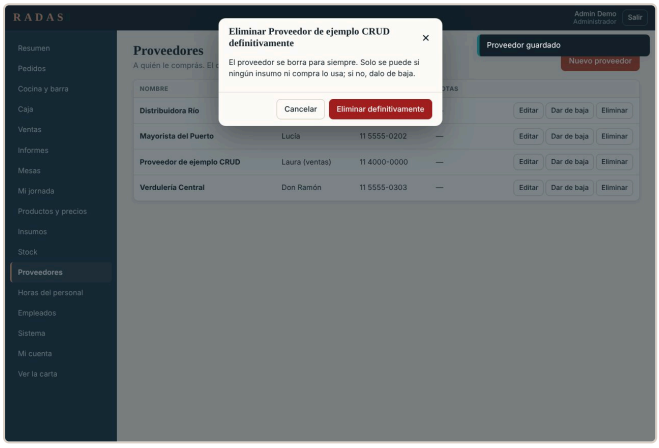


Figura 15.5. Confirmación antes de eliminar.

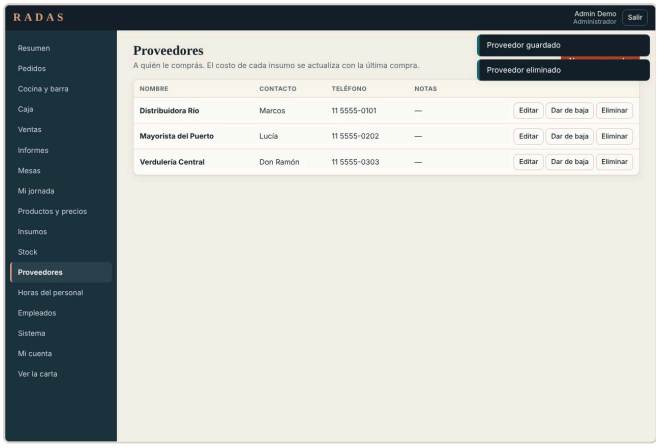


Figura 15.6. Eliminado: ya no está en la lista.

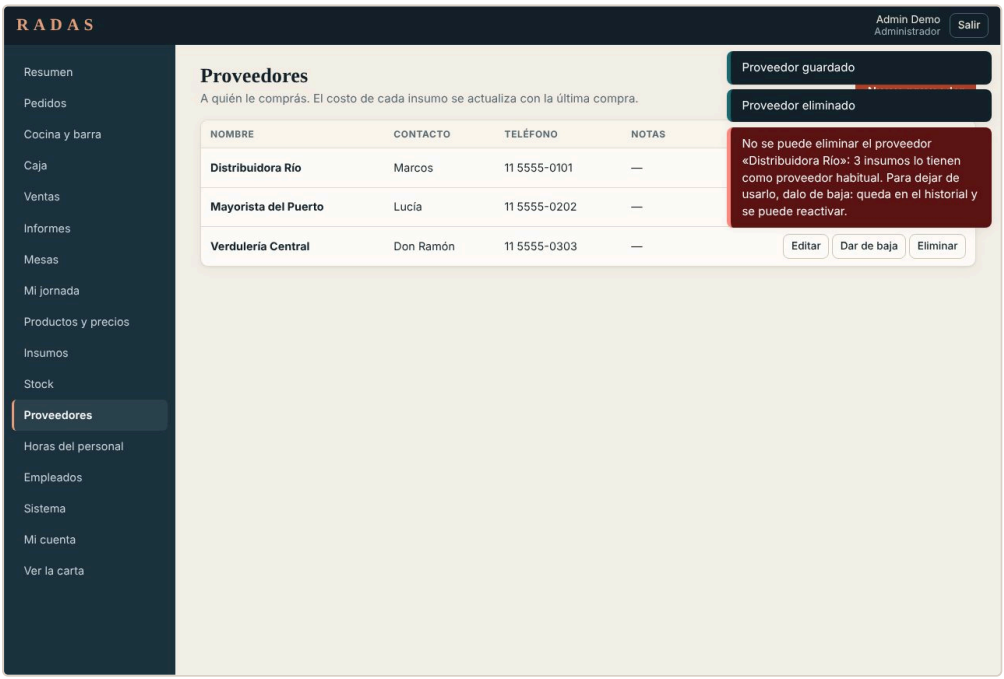


Figura 15.7. Eliminación protegida: «No se puede eliminar el proveedor «Distribuidora Río»: 3 insumos lo tienen como proveedor habitual...».

15.4 Evidencia 2 · Las mismas operaciones contra la API, con las respuestas reales

Traza obtenida ejecutando el flujo contra la API y la base de datos reales (script que reutiliza los mismos auxiliares que las pruebas). Cada fila es una petición HTTP interna con el código y la respuesta que devolvió el sistema.

Operación	Petición	Respuesta
Create	`POST /proveedores`	**201** · {"id": 1, "nombre": "Distribuidora Demo", "contacto": "", "telefono": "11 4000-0000"}
Read	`GET /proveedores`	**200** · lista con 1 proveedor(es)
Create duplicado	`POST /proveedores`	**409** · Ya existe un proveedor con ese nombre
Create con campo obligatorio vacío	`POST /proveedores`	**400** · Datos inválidos · [{"campo": "nombre", "mensaje": "Es obligatorio"}]
Update	`PATCH /proveedores/1`	**200** · {"id": 1, "nombre": "Distribuidora Demo", "contacto": "Laura (ventas)", "telefono": "11 4000-0000"}
Create (insumo con ese proveedor)	`POST /insumos`	**201** · {"id": 3, "nombre": "Ron"}
Delete con datos que lo usan	`DELETE /proveedores/1/definitivo`	**409** · No se puede eliminar el proveedor «Distribuidora Demo»: 1 insumo lo tiene como proveedor habitual. Para dejar de usarlo, dalo de baja: queda en el historial y se puede reactivar.
Delete de un insumo sin historial	`DELETE /insumos/3/definitivo`	**200** · {"id": 3, "nombre": "Ron", "eliminado": true}
Delete ahora sin dependencias	`DELETE /proveedores/1/definitivo`	**200** · {"id": 1, "nombre": "Distribuidora Demo", "eliminado": true}
Delete de algo que ya no existe	`DELETE /proveedores/1/definitivo`	**404** · No se encontró el proveedor: puede que ya se haya eliminado
Delete con rol sin permiso (camarero)	`DELETE /proveedores/1/definitivo`	**403** · No tenés permiso para esta acción

15.5 Evidencia 3 · Prueba automática del CRUD

El archivo `test/eliminar.test.js` verifica estas mismas reglas de forma repetible (ver su ejecución en el capítulo 17): crear → leer → actualizar → eliminar sin historial; eliminar con historial rechazado con 409 y mensaje; borrado de producto junto con su receta; protección de insumos y proveedores con compras; permisos (403/401) y registro en auditoría.

16 Pruebas unitarias

16.1 Qué son y cómo se ejecutan

Una prueba unitaria comprueba una función aislada: no usa base de datos, red ni pantalla, y se ejecuta en milisegundos. En RADAS la lógica de negocio está escrita como **funciones puras** (`src/domain/`), por eso es fácil probarla a fondo.

Dato	Valor
Framework	`node:test` (incluido en Node.js) y `node:assert/strict`. No requiere instalar nada.
Archivos	`test/dominio.test.js` (20), `presentaciones.test.js` (6), `qr.test.js` (6), `informes.test.js` (11): **43 pruebas unitarias**.
Comando	`node --test test/dominio.test.js test/presentaciones.test.js test/qr.test.js test/informes.test.js` (o `npm run test:dominio` para el archivo de dominio).
Resultado de la ejecución	**43 pruebas, 43 aprobadas, 0 fallidas** (evidencia en 16.6).

A continuación se documentan **cuatro pruebas representativas**: qué se probó, el resultado esperado, el resultado obtenido (valores reales devueltos por la función) y el código de la prueba.

16.2 Prueba U1 · Asignación de stock por vencimiento (FEFO)

Qué se probó	La función `asignarFefo(lotes, cantidad)` decide de qué lotes sale lo que se vende: primero el que vence antes, el que no vence va último; informa el faltante si no alcanza y no arrastra errores de coma flotante (0,1 + 0,2).
Por qué importa	Si se descuenta del lote equivocado, se vence mercadería que podía venderse y el costo del plato sale mal. Es la regla que más plata mueve en un bar.
Archivo	`test/dominio.test.js`

Resultado esperado vs. obtenido

Entrada (llamada)	Esperado	Obtenido	Resultado
3 lotes (100 sin vencimiento, 50 vence 20/10, 50 vence 18/10), pedir 120	Lotes 3, 2 y 1: [[3,50],[2,50],[1,20]]; faltante 0	[[3,50],[2,50],[1,20]] · faltante 0	✔ Aprobada
Lote de 0,3 y pedir 0,1 + 0,2	faltante 0	faltante 0	✔ Aprobada
Lote de 0,3 y pedir 1	cantidad 0,3 y faltante 0,7	{"cantidad":0.3,"faltante":0.7}	✔ Aprobada
Sin lotes y pedir 5	sin asignaciones, faltante 5	{"asignaciones":[],"faltante":5}	✔ Aprobada

Código de la prueba

```
test('FEFO: sale primero lo que vence antes y lo que no vence va último', () => {
  const lotes = [
    { id: 1, cantidad_actual: 100, fecha_vencimiento: null, fecha_ingreso: '2026-10-01',
costo_unitario: 1 },
    { id: 2, cantidad_actual: 50, fecha_vencimiento: '2026-10-20', fecha_ingreso: '2026-10-05',
costo_unitario: 2 },
    { id: 3, cantidad_actual: 50, fecha_vencimiento: '2026-10-18', fecha_ingreso: '2026-10-06',
costo_unitario: 3 },
```

```
  ];
  const { asignaciones, faltante } = asignarFefo(lotes, 120);
  assert.deepEqual(asignaciones.map((a) => [a.id_lote, a.cantidad]), [[3, 50], [2, 50], [1, 20]]);
  assert.equal(faltante, 0);
});

test('FEFO: informa el faltante cuando no alcanza y evita errores de coma flotante', () => {
  const lotes = [{ id: 1, cantidad_actual: 0.3, fecha_vencimiento: null, fecha_ingreso: '2026-10-01',
costo_unitario: 1 }];
  assert.equal(asignarFefo(lotes, 0.1 + 0.2).faltante, 0);
  const r = asignarFefo(lotes, 1);
  assert.equal(r.asignaciones[0].cantidad, 0.3);
  assert.equal(r.faltante, 0.7);
  assert.deepEqual(asignarFefo([], 5), { asignaciones: [], faltante: 5 });
});
```

Evidencia de ejecución

```
✔ FEFO: sale primero lo que vence antes y lo que no vence va último (0.330832ms)
✔ FEFO: informa el faltante cuando no alcanza y evita errores de coma flotante (0.161265ms)
```

16.3 Prueba U2 · Costos, márgenes y precios

Qué se probó	Las funciones `costoUnitarioDeCompra`, `costoDeReceta`, `calcularMargen`, `precioParaMargen` y `aplicarPorcentaje`: costo por unidad base de una compra, costo de una receta (marcando como incompleto si un insumo no tiene costo), margen en pesos y porcentaje, precio necesario para un margen deseado y ajuste porcentual con redondeo.
Por qué importa	Es la base de todas las decisiones de precio del dueño; un error de redondeo o una división por cero se traduce en precios mal fijados.
Archivo	`test/dominio.test.js`

Resultado esperado vs. obtenido

Entrada (llamada)	Esperado	Obtenido	Resultado
costoUnitarioDeCompra(18000, 750)	24	24	✔ **Aprobada**
costoUnitarioDeCompra(100, 0) (cantidad cero)	0, sin dividir por cero	0	✔ **Aprobada**
costoDeReceta([30×$24, 30×$10])	{costo 1020, incompleto false}	{"costo":1020,"incompleto":false}	✔ **Aprobada**
costoDeReceta con un insumo sin costo	incompleto = true	true	✔ **Aprobada**
calcularMargen(4000, 1000)	{margen 3000, margen_pct 75}	{"margen":3000,"margen_pct":75}	✔ **Aprobada**
calcularMargen(0, 100).margen_pct (precio 0)	null (no inventa un %)	null	✔ **Aprobada**
precioParaMargen(1000, 75) / (1020, 70) / (1000, 100)	4000 / 3400 / null	4000 / 3400 / null	✔ **Aprobada**
aplicarPorcentaje(7800, +10) / (8200, −5)	8580 / 7790	8580 / 7790	✔ **Aprobada**

Código de la prueba

```
test('costos: costo de receta, margen y precios', () => {
  assert.equal(costoUnitarioDeCompra(18000, 750), 24);
  assert.equal(costoUnitarioDeCompra(100, 0), 0);
  const r = costoDeReceta([{ cantidad: 30, costo_unitario: 24 }, { cantidad: 30, costo_unitario: 10
}]);
  assert.deepEqual(r, { costo: 1020, incompleto: false });
  assert.equal(costoDeReceta([{ cantidad: 30, costo_unitario: 24 }, { cantidad: 5, costo_unitario: 0
```

```
}]).incompleto, true);
  assert.deepEqual(calcularMargen(4000, 1000), { margen: 3000, margen_pct: 75 });
  assert.equal(calcularMargen(0, 100).margen_pct, null);
  assert.equal(precioParaMargen(1000, 75), 4000);
  assert.equal(precioParaMargen(1020, 70), 3400);
  assert.equal(precioParaMargen(1000, 100), null);
  assert.equal(aplicarPorcentaje(7800, 10), 8580);
  assert.equal(aplicarPorcentaje(8200, -5), 7790);
});

import { agruparPorProducto, nivelDeEspera, puedeCambiar, totalDe, transicionValida } from
'../src/domain/pedidos.js';
```

Evidencia de ejecución

```
✔ costos: costo de receta, margen y precios (0.249337ms)
```

16.4 Prueba U3 · Arqueo de caja

Qué se probó	`calcularArqueo`: efectivo esperado = fondo inicial + ventas en efectivo − retiros; la diferencia es contado − esperado.
Por qué importa	Es lo que dice si falta o sobra plata al cerrar la noche. Debe ser exacto al centavo y tolerar los errores de coma flotante.
Archivo	`test/dominio.test.js`

Resultado esperado vs. obtenido

Entrada (llamada)	Esperado	Obtenido	Resultado
ventas en efectivo 10.000, fondo 5.000, retiros 2.000, contado 12.500	esperado 13.000 · diferencia −500	{"efectivo_esperado":13000,"diferencia":-500}	✔ Aprobada
ventas 0,1 + 0,2 y contado 0,3	esperado 0,3 · diferencia 0	{"efectivo_esperado":0.3,"diferencia":0}	✔ Aprobada
sin haber contado todavía	diferencia null	null	✔ Aprobada

Código de la prueba

```
test('arqueo: efectivo esperado = fondo + ventas en efectivo − retiros', () => {
  assert.deepEqual(calcularArqueo({ ventas_efectivo: 10000, fondo_inicial: 5000, retiros: 2000,
efectivo_contado: 12500 }), { efectivo_esperado: 13000, diferencia: -500 });
  assert.deepEqual(calcularArqueo({ ventas_efectivo: 0.1 + 0.2, efectivo_contado: 0.3 }), {
efectivo_esperado: 0.3, diferencia: 0 });
  assert.equal(calcularArqueo({ ventas_efectivo: 100 }).diferencia, null);
});
```

Evidencia de ejecución

```
✔ arqueo: efectivo esperado = fondo + ventas en efectivo − retiros (0.17156ms)
```

16.5 Prueba U4 · Validación de números escritos a la argentina

Qué se probó	`aNumero` (y `numeroDe`) interpretan lo que escribe el usuario en un formulario: el punto separa miles y la coma los decimales; un punto con otra cantidad de cifras se toma como decimal; el texto no numérico devuelve `null`.

Por qué importa	Es una validación de entrada: si «5.000» se leyera como 5, un precio o una cantidad saldrían mil veces mal.
Archivo	`test/dominio.test.js`

Resultado esperado vs. obtenido

Entrada (llamada)	Esperado	Obtenido	Resultado
aNumero('5.000')	5000	5000	✔ Aprobada
aNumero('1.500,50')	1500.5	1500.5	✔ Aprobada
aNumero('1500,5')	1500.5	1500.5	✔ Aprobada
aNumero('1500.5')	1500.5	1500.5	✔ Aprobada
aNumero('0.500')	0.5	0.5	✔ Aprobada
aNumero('1.234.567')	1234567	1234567	✔ Aprobada
aNumero('abc')	null	null	✔ Aprobada

Código de la prueba

```
test('números escritos a la argentina: el punto separa miles y la coma los decimales', () => {
  assert.equal(aNumero('5.000'), 5000);
  assert.equal(aNumero('1.500,50'), 1500.5);
  assert.equal(aNumero('1500,5'), 1500.5);
  assert.equal(aNumero('1500.5'), 1500.5);   // estilo inglés con decimales
  assert.equal(aNumero('0.500'), 0.5);       // empieza con cero: es un decimal
  assert.equal(aNumero('1.234.567'), 1234567);
  assert.equal(aNumero('12.5'), 12.5);
  assert.equal(aNumero('abc'), null);
  assert.equal(aNumero(''), null);
  assert.ok(Number.isNaN(numeroDe('1,2,3')) && numeroDe('') === 0 && numeroDe(' 7.000 ') === 7000);
});

import { aBase, costoDeEnvase, desglosar, equivalencia, normalizarPresentaciones, principalDe } from
'../src/domain/presentaciones.js';
```

Evidencia de ejecución

```
✔ números escritos a la argentina: el punto separa miles y la coma los decimales (0.44335ms)
```

16.6 Evidencia de ejecución de las 43 pruebas unitarias

```
$ node --test --test-reporter=spec test/dominio.test.js test/presentaciones.test.js test/qr.test.js
test/informes.test.js
✔ fechas: día local, inicio de día y aritmética (2.593386ms)
✔ horas: jornada nocturna que cruza la medianoche (0.266802ms)
✔ horas: fichaje abierto cuenta hasta ahora y detecta olvido de egreso (0.115827ms)
✔ horas: resumen por empleado, turno y día de inicio (1.758131ms)
✔ FEFO: sale primero lo que vence antes y lo que no vence va último (0.330832ms)
✔ FEFO: informa el faltante cuando no alcanza y evita errores de coma flotante (0.161265ms)
✔ alertas: reponer, por vencer, vencidos y sobrestock (0.477026ms)
✔ costos: costo de receta, margen y precios (0.249337ms)
✔ pedidos: pasos válidos entre estados (0.279963ms)
✔ pedidos: quién puede cada cambio (1.445371ms)
✔ pedidos: total ignora lo anulado; los redondeos no acumulan centavos (0.256558ms)
✔ pantalla de cocina: agrupa lo igual, separa por nota y por estado, y ordena por antigüedad
(0.267254ms)
✔ pantalla de cocina: la espera pasa a amarillo y rojo según el destino (0.072328ms)
✔ jornada comercial: de 06:00 a 06:00, así lo vendido de madrugada va a la noche anterior (0.899716ms)
```

```
✔ arqueo: efectivo esperado = fondo + ventas en efectivo − retiros (0.17156ms)
✔ resumen de ventas: sin ventas no divide por cero; un producto sin costo no inventa margen
(1.092705ms)
✔ números escritos a la argentina: el punto separa miles y la coma los decimales (0.44335ms)
✔ presentaciones: de envases a la unidad base y de vuelta (0.173286ms)
✔ presentaciones: el texto de equivalencia solo aparece cuando hay al menos un envase entero
(0.171092ms)
✔ presentaciones: se normalizan los nombres y siempre hay una sola principal (0.26079ms)
✔ consumo real vs teórico: ventas, anulaciones, mermas y conteo, en cantidades y pesos (2.929849ms)
✔ consumo: un insumo sin costo cargado se avisa y no inventa valores (0.247599ms)
✔ stock valorizado: por lote, con faltante de carga aparte y sin contar lo que no hay (0.435356ms)
✔ mermas en pesos: por motivo y por insumo, y qué parte de las ventas representan (0.323368ms)
✔ compras: lo que está bajo el mínimo se pide hasta el máximo y se agrupa por proveedor (0.541134ms)
✔ compras: redondea a envases enteros y usa el ritmo de consumo para avisar antes de llegar al mínimo
(0.181689ms)
✔ compras: sin proveedor queda al final y sin costo se avisa; el texto para WhatsApp sale armado
(0.375395ms)
✔ rentabilidad: aporte a la ganancia, clases A/B/C y cuadrante de ingeniería de menú (0.41781ms)
✔ rentabilidad sin ventas no rompe (0.269307ms)
✔ comparar períodos: variaciones, venta por noche aunque duren distinto, y productos nuevos o sin
ventas (0.787407ms)
✔ comparar con un período sin datos no divide por cero (0.258195ms)
✔ un insumo recién creado no tiene presentaciones (86.24523ms)
✔ se cargan varias presentaciones; la primera queda como principal si nadie marca otra (1.894472ms)
✔ se puede elegir la principal, y reemplazar o vaciar la lista (1.11162ms)
✔ el mismo nombre se puede usar en insumos distintos, pero no repetir dentro de uno (1.336848ms)
✔ validaciones: cantidades, nombres, dos principales, demasiadas y insumo inexistente (2.822054ms)
✔ permisos: solo el administrador las cambia; el personal de barra y cocina las ve sin costos ni stock
(1.029861ms)
✔ resultado fijo para textos ya verificados con un decodificador real (41.116886ms)
✔ el tamaño crece con el texto y respeta la capacidad de cada versión (28.41073ms)
✔ tiene los tres buscadores, las líneas de sincronía y el módulo oscuro fijo (14.507998ms)
✔ es determinista y distingue textos distintos (19.836166ms)
✔ acepta caracteres fuera de ASCII (se codifican en UTF-8) (2.402084ms)
✔ el SVG tiene zona de silencio, es autocontenido y no lleva scripts (2.602816ms)
ℹ tests 43
ℹ suites 0
ℹ pass 43
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 453.716373
```

16.7 Resultado de toda la batería en los dos motores

Las 187 pruebas (unitarias e integración) pasan sobre SQLite en memoria y contra **PostgreSQL 16 real**:

```
$ npm test                       # SQLite en memoria
# tests 187
# pass 187
# fail 0
# cancelled 0
# skipped 0
# duration_ms 8150.413863

$ RADAS_PSQL="-h /tmp/pgsock -p 5433 -U postgres -d radas_test" npm test    # PostgreSQL 16.15
# tests 187
# pass 187
# fail 0
# cancelled 0
# skipped 0
# duration_ms 14207.123925
```

17 Pruebas de integración

17.1 Qué son y cómo se ejecutan

Una prueba de integración verifica que varios módulos funcionen **juntos**: aquí, la **API real** (autenticación, permisos, validación), la **lógica de negocio** y una **base de datos real** (SQLite en memoria, o PostgreSQL 16 con `RADAS_PSQL`). Cada prueba empieza con una base vacía, crea sus datos a través de la propia API y comprueba el resultado leyendo de nuevo por la API.

Dato	Valor
Archivos	`catalogo` (16), `eliminar` (5), `pedidos` (15), `caja-canales` (7), `mover` (9), `respaldo` (10) y otros: 144 pruebas de integración en total.
Comando	`node --test --test-concurrency=1 test/pedidos.test.js test/eliminar.test.js …` o `npm test` para todo.
Además	Recorrido de navegador (Chromium) de **62 pasos** que atraviesa interfaz → API → base: `test/e2e/navegador.mjs`.

17.2 Prueba I1 · Un pedido impacta en el stock, la mesa y el cobro

Qué se probó	Que la creación de un pedido desde la interfaz (`POST /pedidos`) se refleje correctamente en **varias tablas a la vez**: crea el pedido y sus líneas con el precio fijado, descuenta del stock cada insumo de la receta, ocupa la mesa y, al cobrar, cierra la cuenta y libera la mesa.
Módulos que interactúan	Autenticación y permisos → validación → pedidos → stock (FEFO) → mesas → base de datos.
Archivo y casos	`test/pedidos.test.js`: «una cuenta de mesa: total, mesa ocupada y stock descontado por receta», «cobro: total, medio de pago, cuenta cerrada y mesa liberada», «no se cobra con pedidos sin terminar», «dos cuentas abiertas a la vez en la misma mesa: gana una sola».

Pasos realizados

1. Se cargan los datos de partida por la API: insumos **Gin** (1000 ml) y **Tónica** (2000 ml) con su compra, el producto **Gin tonic** ($ 8.000; receta: 50 ml de gin + 150 ml de tónica) y la mesa **M1**.
2. El camarero abre una cuenta en M1 con **2 Gin tonic** y la nota «sin hielo».
3. Se consulta el stock y el estado de la mesa por la API.
4. Se intenta abrir otra cuenta en la misma mesa y cobrar con líneas sin terminar.
5. Cocina/barra marca las líneas como listas y entregadas; el cajero cobra con tarjeta.
6. Se consulta de nuevo la mesa.

Resultado esperado vs. obtenido

Comprobación	Esperado	Obtenido	
Stock de Gin tras el pedido (2 × 50 ml)	1000 → 900	1000 → 900	✔
Stock de Tónica tras el pedido (2 × 150 ml)	2000 → 1700	2000 → 1700	✔
Total de la cuenta (2 × $ 8.000)	16000	16000	✔
Línea creada (producto, cantidad, precio, estado, nota)	Gin tonic · 2 · 8000 · pendiente · sin hielo	Gin tonic · 2 · 8000 · pendiente · sin hielo	✔
Estado de la mesa tras abrir la cuenta	ocupada	ocupada	✔

Comprobación	Esperado	Obtenido	
Segunda cuenta en la misma mesa	409 «Esa mesa ya tiene una cuenta abierta»	409 · Esa mesa ya tiene una cuenta abierta	✔
Cobrar con líneas sin terminar	409 con mensaje	409 · Todavía hay pedidos sin terminar. Esperá a que estén listos o anulalos antes de cobrar	✔
Estado de la cuenta tras cobrar	cerrado	cerrado	✔
Estado de la mesa tras cobrar	libre	libre	✔

Traza de las peticiones (respuestas reales)

Petición	Código	Respuesta (resumen)
`POST /insumos`	201	{"id": 1, "nombre": "Gin"}
`POST /insumos`	201	{"id": 2, "nombre": "Tónica"}
`POST /stock/ingresos`	201	{}
`POST /stock/ingresos`	201	{}
`POST /productos`	201	{"id": 1, "nombre": "Gin tonic"}
`POST /mesas`	201	{"id": 1, "nombre": "M1", "estado": "libre&qu
`POST /pedidos`	201	cuenta #1 · estado abierto · total 16000
`POST /pedidos`	409	Esa mesa ya tiene una cuenta abierta
`POST /pedidos/1/cerrar`	409	Todavía hay pedidos sin terminar. Esperá a que estén listos o anulalos antes de cobrar
`PATCH /pedido-items/estado`	200	{"estado": "listo"}
`PATCH /pedido-items/estado`	200	{"estado": "entregado"}
`POST /pedidos/1/cerrar`	200	cuenta #1 · estado cerrado · total 16000

Código de la prueba (fragmento)

```
test('una cuenta de mesa: total, mesa ocupada y stock descontado por receta', async () => {
  const antesGin = await stockDe('Gin');
  const p = await abrir('M1', [linea('Gin tonic', 2), linea('Tostada', 1, 'sin sal')]);
  assert.equal(p.estado, 'abierto');
  assert.equal(p.mesa, 'M1');
  assert.equal(p.total, 2 * 8000 + 5000);
  assert.equal(p.items.length, 2);
  assert.ok(p.items.every((i) => i.estado === 'pendiente'));
  assert.equal(p.items[1].nota, 'sin sal');
  assert.equal(await stockDe('Gin'), antesGin - 100);              // 2 × 50 ml
  assert.equal(await stockDe('Tónica'), 2000 - 300);
  assert.equal(await stockDe('Pan'), 20 - 2);
  assert.equal(await mesaEstado('M1'), 'ocupada');
  ids.pedidoM1 = p.id;
});

test('cobro: total, medio de pago, cuenta cerrada y mesa liberada', async () => {
  const cerrar = (rol, id, medio) => llamar('POST', `/pedidos/${id}/cerrar`, rol, { body: {
medio_pago: medio }, ahora: min(10) });
  assert.equal((await cerrar('camarero', ids.pedidoM3, 'bitcoin')).status, 400);
  const c = esperar(await cerrar('cajero', ids.pedidoM3, 'tarjeta'), 200);
  assert.equal(c.estado, 'cerrado');
  assert.equal(c.total, 13000);
  assert.equal(c.medio_pago, 'tarjeta');
```

```
  assert.equal(await mesaEstado('M3'), 'libre');
  assert.equal((await cerrar('cajero', ids.pedidoM3, 'efectivo')).status, 409);   // ya cerrada
```

Evidencia de ejecución

```
✔ una cuenta de mesa: total, mesa ocupada y stock descontado por receta (117.213937ms)
✔ cobro: total, medio de pago, cuenta cerrada y mesa liberada (3.803305ms)
✔ no se cobra con pedidos sin terminar; lo que está listo se da por entregado al cobrar (2.866657ms)
✔ dos cuentas abiertas a la vez en la misma mesa: gana una sola (0.862402ms)
✔ pedidos simultáneos no pierden stock (2.627738ms)
```

17.3 Prueba I2 · CRUD completo: crear, leer, actualizar y eliminar con integridad

Qué se probó	Que las cuatro operaciones del CRUD funcionen de punta a punta (API → validación → lógica → base) y que la eliminación respete la integridad: se puede borrar lo que no tiene historial y se rechaza, con un mensaje claro, lo que sí lo tiene. Además, permisos y auditoría.
Archivo y casos	`test/eliminar.test.js` (5 casos) y `test/catalogo.test.js` (16 casos: duplicados, validaciones, recetas, precios).

Pasos y resultado (esperado vs. obtenido)

#	Paso	Esperado	Obtenido	
1	**Create** `POST /proveedores`	201	201 · {"id": 1, "nombre": "Distribuidora Demo", "contacto": ""}	✔
2	**Read** `GET /proveedores`	200	200 · 1 proveedor(es)	✔
3	**Create duplicado** `POST /proveedores`	409 «Ya existe un proveedor con ese nombre»	409 · Ya existe un proveedor con ese nombre	✔
4	**Create con campo obligatorio vacío** `POST /proveedores`	400 «Es obligatorio» en el campo nombre	400 · Datos inválidos (nombre: Es obligatorio)	✔
5	**Update** `PATCH /proveedores/1`	200 con el contacto modificado	200 · {"id": 1, "nombre": "Distribuidora Demo", "contacto": "Laura (ventas)"}	✔
6	**Create (insumo con ese proveedor)** `POST /insumos`	201	201 · {"id": 3, "nombre": "Ron"}	✔
7	**Delete con datos que lo usan** `DELETE /proveedores/1/definitivo`	409 que explica el motivo y sugiere la baja	409 · No se puede eliminar el proveedor «Distribuidora Demo»: 1 insumo lo tiene como proveedor habitual. Para dejar de usarlo, dalo de baja: queda en el historial y se puede reactivar.	✔
8	**Delete de un insumo sin historial** `DELETE /insumos/3/definitivo`	200 · eliminado	200 · {"id": 3, "nombre": "Ron", "eliminado": true}	✔
9	**Delete ahora sin dependencias** `DELETE /proveedores/1/definitivo`	200 · eliminado	200 · {"id": 1, "nombre": "Distribuidora Demo", "eliminado": true}	✔
10	**Delete de algo que ya no existe** `DELETE /proveedores/1/definitivo`	404	404 · No se encontró el proveedor: puede que ya se haya eliminado	✔

#	Paso	Esperado	Obtenido	
11	**Delete con rol sin permiso (camarero)** `DELETE /proveedores/1/definitivo`	403	403 · No tenés permiso para esta acción	✔

Código de la prueba (fragmento)

```
  assert.ok(esperar(await llamar('GET', '/mesas', admin), 200).mesas.some((m) => m.id ===
creada.id));

  const editada = esperar(await llamar('PATCH', `/mesas/${creada.id}`, admin, { capacidad: 6 }),
200);
  assert.equal(editada.capacidad, 6);

  const r = esperar(await llamar('DELETE', `/mesas/${creada.id}/definitivo`, admin), 200);
  assert.deepEqual([r.eliminado, r.nombre], [true, 'Prueba 1']);
  assert.ok(!esperar(await llamar('GET', '/mesas', admin), 200).mesas.some((m) => m.id ===
creada.id));
  assert.equal((await llamar('DELETE', `/mesas/${creada.id}/definitivo`, admin)).status, 404); // ya
no existe
});

test('mesa: con pedidos en el historial no se elimina (409 con mensaje claro) pero sí se da de baja',
async () => {
  ids.mesa = esperar(await llamar('POST', '/mesas', admin, { nombre: 'Con historial', zona:
'exterior', capacidad: 2 }), 201).id;
  ids.plato = esperar(await llamar('POST', '/productos', admin, { nombre: 'Tostado', categoria:
'Comida', destino: 'cocina', precio: 5000, receta: [] }), 201).id;
  esperar(await llamar('POST', '/pedidos', camarero, { canal: 'mesa', id_mesa: ids.mesa, items: [{
id_producto: ids.plato, cantidad: 1 }] }), 201);

  assert.equal(vendido.status, 409);
  assert.match(vendido.body.error, /se vendió o pidió en 1 ocasión/);
  assert.equal(esperar(await llamar('GET', `/productos/${ids.plato}`, admin), 200).nombre,
'Tostado'); // sigue ahí
});

test('insumo con compras registradas y proveedor con insumos: protegidos', async () => {
  const prov = esperar(await llamar('POST', '/proveedores', admin, { nombre: 'Proveedor de prueba'
}), 201);
  const ins = esperar(await llamar('POST', '/insumos', admin, { nombre: 'Ron', unidad: 'ml',
id_proveedor: prov.id }), 201);
  esperar(await llamar('POST', '/stock/ingresos', admin, { id_insumo: ins.id, cantidad: 1000,
costo_total: 9000, id_proveedor: prov.id }), 201);

  const a = await llamar('DELETE', `/insumos/${ins.id}/definitivo`, admin);
  assert.equal(a.status, 409);
  assert.match(a.body.error, /compra o lote/);
  const b = await llamar('DELETE', `/proveedores/${prov.id}/definitivo`, admin);
  assert.equal(b.status, 409);
  assert.match(b.body.error, /Proveedor de prueba/);

  const suelto = esperar(await llamar('POST', '/proveedores', admin, { nombre: 'Sin movimientos' }),
201);
  esperar(await llamar('DELETE', `/proveedores/${suelto.id}/definitivo`, admin), 200);
  assert.ok(!esperar(await llamar('GET', '/proveedores', admin), 200).some((p) => p.id ===
suelto.id));
```

Evidencia de ejecución

```
✔ mesa: crear → leer → actualizar → eliminar definitivamente (sin historial) (95.958923ms)
✔ mesa: con pedidos en el historial no se elimina (409 con mensaje claro) pero sí se da de baja
(4.258773ms)
✔ producto: sin ventas se elimina (con su receta); con ventas queda protegido (5.121302ms)
```

```
✔ insumo con compras registradas y proveedor con insumos: protegidos (17.247903ms)
✔ eliminar definitivamente es solo del administrador y queda en la auditoría (2.119951ms)
✔ proveedores: alta, duplicado y edición (111.870223ms)
✔ insumos: validaciones, duplicado y proveedor inexistente (3.859382ms)
✔ dar de baja: un insumo en uso no se puede; un producto sale de la carta pero se conserva
(4.576435ms)
```

17.4 Prueba I3 · Cierre de caja por canal

Qué se probó	La interacción entre pedidos, cobros, caja y base de datos: una cuenta abierta de la barra frena solo el cierre de la barra; el cierre del salón calcula el arqueo con fondo y retiros; con el salón cerrado no se cobra en mesas pero sí en la barra; reabrir libera solo ese canal.
Archivo	`test/caja-canales.test.js` (7 casos).
Esperado	Cierre del salón: ventas $ 10.000, efectivo esperado $ 10.500 (fondo 1.000 + ventas 10.000 − retiros 500), contado $ 10.400, diferencia −$ 100, 1 cuenta; rechazo 409 «cuenta abierta» al cerrar la barra con una cuenta sin cobrar.
Obtenido	Todas las aserciones se cumplieron (ver ejecución).

```
test('una cuenta abierta de la barra frena el cierre de la barra, pero no el del salón', async () =>
{
  const abierta = await abrir({ canal: 'barra', etiqueta: 'Caro' }, [L(ids.Trago)]);
  const j = await jornada();
  assert.equal(j.por_caja.barra.abiertas.length, 1);
  assert.equal(j.por_caja.mesa.abiertas.length, 0);
  const rechazo = await cerrar('barra', { efectivo_contado: 5000 });
  assert.equal(rechazo.status, 409);
  assert.match(rechazo.body.error, /cuenta abierta/);
  const ok = esperar(await cerrar('mesa', { fondo_inicial: 1000, retiros: 500, efectivo_contado:
10400 }), 201);
  assert.equal(ok.canal, 'mesa');
  assert.deepEqual([ok.ventas_total, ok.ventas_efectivo, ok.efectivo_esperado, ok.diferencia,
ok.cuentas], [10000, 10000, 10500, -100, 1]);
  await entregar(abierta);
  esperar(await cobrar(abierta, 'efectivo'), 200); // la barra sigue abierta: se puede cobrar
});
```

```
✔ la jornada separa lo cobrado en el salón y en la barra (121.360609ms)
✔ una cuenta abierta de la barra frena el cierre de la barra, pero no el del salón (11.217061ms)
✔ con el salón cerrado, no se cobra en una mesa; en la barra sí (4.335895ms)
✔ no se mezclan: con el salón cerrado no se cierra «todo junto», y al cerrar la barra la noche queda
cerrada (12.819581ms)
✔ reabrir una caja deja cobrar de nuevo solo en ese canal (6.291674ms)
✔ cerrar «todo junto» sigue funcionando y bloquea los cierres por caja (20.790444ms)
✔ validaciones y la base garantiza una sola caja por jornada y canal (0.788948ms)
```

17.5 Prueba I4 · Recorrido completo en un navegador real (extremo a extremo)

El archivo `test/e2e/navegador.mjs` usa Playwright con Chromium para operar la **interfaz como lo haría una persona** (ingresar con cada rol, tomar pedidos, preparar, cobrar, imprimir el ticket, cerrar caja, crear copias, el CRUD de proveedores, etc.) contra el servidor y la base reales, y comprueba lo que aparece en pantalla. Abarca 62 pasos; también verifica que ninguna pantalla desborde horizontalmente a 768, 1024 y 390 px de ancho y que no haya errores de JavaScript en la consola.

```
$ node test/e2e/navegador.mjs
  ✓ 1. la carta para clientes (/carta) se ve sin sesión, sin scripts y sin ninguna salida hacia el
```

```
login
  ✓ 2. una clave incorrecta muestra el error y no entra
  ✓ 3. el administrador entra y ve el resumen con alertas de stock
  ✓ 4. el administrador edita el precio final de un plato y se guarda
  ✓ 5. ajuste en bloque del +10 % en una categoría con vista previa
  ✓ 6. la vista previa por margen deseado calcula sin aplicar
  … (pasos 7 a 47 omitidos en este extracto)
  ✓ 48. sistema: se crea una copia de seguridad y se descarga
  ✓ 49. auditoría: queda registrado quién creó y descargó la copia, y se puede filtrar
  ✓ 50. sistema: el QR de la carta se genera en pantalla
  ✓ 51. pedidos: se pasa un producto suelto a una mesa libre (se abre una cuenta nueva)
  ✓ 52. pedidos: la cuenta se pasa a una mesa ocupada y se junta; el origen queda libre
  ✓ 53. caja: se cierra el salón por separado y la barra sigue cobrando
  ✓ 54. informes: rentabilidad, comparación, consumo, stock/mermas y compras sugeridas cargan
  ✓ 55. CRUD crear/leer: el proveedor nuevo aparece en la lista
  ✓ 56. validación: un nombre repetido se rechaza con un mensaje claro
  ✓ 57. CRUD actualizar: el cambio se ve en la lista
  ✓ 58. CRUD eliminar: pide confirmación, cancelar no borra y confirmar elimina
  ✓ 59. integridad: un proveedor en uso no se elimina y el mensaje explica por qué
  ✓ 60. responsive: 16 pantallas × 3 tamaños (tablet vertical/horizontal y celular) sin scroll
horizontal
  ✓ 61. carta de clientes: se ve bien en el celular
  ✓ 62. en el celular no hay scroll horizontal de página

Sin errores de JavaScript en el navegador.
(62 de 62 pasos aprobados)
```

Las capturas de las figuras de los capítulos 13 y 15 provienen de esta misma automatización.

17.6 Evidencia de ejecución de las pruebas de integración (62 casos de los archivos documentados)

```
$ node --test --test-concurrency=1 --test-reporter=spec test/pedidos.test.js test/eliminar.test.js
test/caja-canales.test.js test/mover.test.js test/respaldo.test.js test/catalogo.test.js
✔ la jornada separa lo cobrado en el salón y en la barra (121.360609ms)
✔ una cuenta abierta de la barra frena el cierre de la barra, pero no el del salón (11.217061ms)
✔ con el salón cerrado, no se cobra en una mesa; en la barra sí (4.335895ms)
✔ no se mezclan: con el salón cerrado no se cierra «todo junto», y al cerrar la barra la noche queda
cerrada (12.819581ms)
✔ reabrir una caja deja cobrar de nuevo solo en ese canal (6.291674ms)
✔ cerrar «todo junto» sigue funcionando y bloquea los cierres por caja (20.790444ms)
✔ validaciones y la base garantiza una sola caja por jornada y canal (0.788948ms)
✔ proveedores: alta, duplicado y edición (111.870223ms)
✔ insumos: validaciones, duplicado y proveedor inexistente (3.859382ms)
✔ insumos: el máximo se valida contra el mínimo también al editar (1.926417ms)
✔ costo por proveedor: cada compra fija el costo unitario y se pueden comparar proveedores
(22.360678ms)
✔ productos: costo de receta, margen y receta con costo por línea (2.187261ms)
✔ productos: un insumo sin costo marca el costo como incompleto; sin receta no hay margen (1.114645ms)
✔ productos: validaciones de receta y nombre duplicado (1.516191ms)
✔ receta: se puede reemplazar y el costo se recalcula (1.992128ms)
✔ editar un producto no cambia su precio: el precio tiene su propio camino (0.944357ms)
✔ precio: cambio individual y valores inválidos (3.22658ms)
✔ precios finales de varios platos: todo o nada (1.304258ms)
✔ ajuste por porcentaje: la vista previa no cambia nada y aplicar sí (2.357268ms)
✔ ajuste por margen: fija el precio para llegar al margen pedido y avisa lo que no puede calcular
(3.672318ms)
✔ el personal no ve costos ni márgenes; el administrador sí (1.087531ms)
✔ dar de baja: un insumo en uso no se puede; un producto sale de la carta pero se conserva
(4.576435ms)
✔ la carta agrupa por categoría (0.209697ms)
✔ mesa: crear → leer → actualizar → eliminar definitivamente (sin historial) (95.958923ms)
✔ mesa: con pedidos en el historial no se elimina (409 con mensaje claro) pero sí se da de baja
```

```
(4.258773ms)
✔ producto: sin ventas se elimina (con su receta); con ventas queda protegido (5.121302ms)
✔ insumo con compras registradas y proveedor con insumos: protegidos (17.247903ms)
✔ eliminar definitivamente es solo del administrador y queda en la auditoría (2.119951ms)
✔ pasar la cuenta a una mesa libre: cambia de mesa, la anterior se libera y la nueva se ocupa
(137.727586ms)
✔ si la mesa de destino ya tiene cuenta, pide confirmar la unión; al unir se suman las líneas
(2.495372ms)
✔ validaciones: misma mesa, mesa inexistente, cuenta cerrada, rol sin permiso (3.457293ms)
✔ una cuenta de la barra se puede pasar a una mesa (1.885859ms)
✔ dividir: pasar líneas a una mesa libre abre una cuenta nueva; la de origen sigue con el resto
(2.821622ms)
✔ pasar líneas a la cuenta que ya tiene otra mesa las suma; si el origen se queda vacío se cancela y
se libera (1.499716ms)
✔ el stock y las anulaciones siguen funcionando con las líneas movidas (6.344296ms)
✔ mover-items: validaciones (3.266371ms)
✔ queda constancia en la auditoría (0.471549ms)
✔ una cuenta de mesa: total, mesa ocupada y stock descontado por receta (117.213937ms)
✔ el precio y el costo quedan fijados: cambiar la carta después no altera lo pedido (1.923711ms)
✔ validaciones al abrir una cuenta (3.069899ms)
✔ permisos: host y cocina no toman pedidos; cajero y barra sí (6.67923ms)
✔ agregar líneas a una cuenta abierta descuenta stock de nuevo (1.864511ms)
✔ KDS: cada destino ve solo lo suyo, en orden, y el camarero no entra (1.800071ms)
✔ estados: quién puede cada paso, atomicidad y deshacer (5.61101ms)
✔ cobro: total, medio de pago, cuenta cerrada y mesa liberada (3.803305ms)
✔ no se cobra con pedidos sin terminar; lo que está listo se da por entregado al cobrar (2.866657ms)
✔ anular: pendiente devuelve el stock exacto; preparando solo el administrador y sin devolver
(11.302814ms)
✔ un entregado no se anula (2.131495ms)
✔ sin stock se puede vender (S-16): avisa el faltante y la anulación mantiene lotes = stock
(3.35107ms)
✔ pedidos simultáneos no pierden stock (2.627738ms)
✔ dos cuentas abiertas a la vez en la misma mesa: gana una sola (0.862402ms)
✔ una mesa con cuenta abierta no se puede liberar a mano (0.718292ms)
✔ el respaldo cubre todas las tablas del esquema (salvo las excluidas a propósito) (239.672053ms)
✔ ida y vuelta: crear copia, ensuciar la base, restaurar y quedar idéntico (28.362428ms)
✔ restaurar es todo o nada: una copia dañada no deja la base a medias (23.143901ms)
✔ interpretar rechaza archivos que no son copias o están incompletos (1.13ms)
✔ restaurar tolera columnas que la base ya no tiene (19.612359ms)
✔ dos copias en el mismo segundo no se pisan, y se conservan solo las últimas N (25.582674ms)
✔ API: solo el administrador; crear, listar y descargar dejan rastro en la auditoría (10.912682ms)
✔ API: no se puede pedir un archivo fuera de la carpeta de copias (1.424244ms)
✔ copia automática: crea, depura y avisa a la copia externa; si el envío falla queda registrado
(97.396409ms)
✔ script de terminal: crear, listar, verificar y restaurar (pide confirmación) (38.287376ms)
ℹ tests 62
ℹ suites 0
ℹ pass 62
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 1760.144741
```

18 Entregables, verificación y limitaciones conocidas

18.1 Lista de entregables

Entregable	Dónde está
Repositorio Git actualizado con la versión final	https://github.com/rodgpinto/PPIV-ProyectoIntegrador — rama `main`
Proyecto completo comprimido	Archivo `.zip` adjunto (código, `docs/`, `sql/`, pruebas y este PDF)
Documento PDF con toda la documentación	Este documento
Manual Técnico	Capítulos 3 a 10 (con la instalación detallada en el capítulo 2)
Manual de Usuario	Capítulos 11 a 14
Evidencia de pruebas unitarias	Capítulo 16 (43 pruebas; 4 documentadas en detalle)
Evidencia de pruebas de integración	Capítulo 17 (4 documentadas en detalle, más el recorrido de navegador de 62 pasos)
Base de datos o scripts de creación	`sql/schema.sql` (estructura) y `sql/datos-ejemplo.sql` (datos); el DER en el capítulo 6
Aplicación desplegada	**https://radas.onrender.com**

18.2 Qué se verificó y qué no

Verificado	No verificado / limitaciones
• 187 pruebas automáticas, sobre SQLite en memoria y sobre PostgreSQL 16 real • Recorrido de navegador de 62 pasos con Chromium (todos los roles, ticket, caja, CRUD) • Sin scroll horizontal a 768, 1024 y 390 px de ancho • Carga y verificación de `sql/schema.sql` y `sql/datos-ejemplo.sql` en PostgreSQL 16 • Despliegue real en Render + Neon: el driver `pg` arrancó contra PostgreSQL real y se restauró una copia de datos de ejemplo • Instalación local (`npm install`, `npm run seed`, `npm run dev`) en las computadoras del equipo	• La instalación local con PGlite no se pudo reproducir en el entorno donde se generó este documento (sin acceso a internet para `npm install`); la ejecutó el equipo en sus equipos • No se probó la interfaz en Safari/iOS ni con muchos usuarios simultáneos • Docker y los despliegues en servidor propio (`docs/DESPLIEGUE.md`) están documentados pero sin probar • El plan gratuito de Render se duerme tras 15 minutos sin uso y su disco es efímero: es un entorno de demostración, no de producción • No emite facturas: el ticket es un comprobante interno no fiscal; la facturación electrónica es solo un diseño (`docs/FACTURACION.md`)

18.3 Guías complementarias del repositorio

Archivo	Contenido
`README.md`	Presentación del proyecto, usuarios de ejemplo, permisos por rol y alcance

Archivo	Contenido
`docs/DESPLIEGUE.md`	Cómo está publicado (Render, Neon, GitHub Actions, UptimeRobot) y opciones de servidor propio
`docs/COPIAS.md`	Copias de seguridad y restauración
`docs/MANTENIMIENTO.md`	Guía para desarrolladores: estructura del código, cómo agregar rutas, tablas y pantallas
`docs/FACTURACION.md`	Diseño de la facturación electrónica (ARCA), con diagramas